Experimentelle Analyse Subgraph-SAT-Solver auf Boolean Circuits

Stephan Epp

M.Sc.

10. Oktober 2026

Zusammenfassung

Wir berichten über 288 Benchmark-Läufe des Subgraph-SAT-Solvers auf drei Familien von Boolean Circuits: synthetischen Target-AND-Circuits (4 bis 20 Eingänge), Target-AND-Circuits mit zufälligen Zwischenschichten (Tiefe 1 bis 12) und den zwölf ISCAS'85-Benchmarks mit bis zu 3 512 Gates und 10 690 CNF-Klauseln. Die Läufe verteilen sich auf vier Experiment-Suites (Quick, Standard, Full, Performance). Die gemessenen Laufzeiten liegen zwischen 6,02 und 40,27 ms (Mittelwert 21,81 ms, Standardabweichung 7,25 ms). Sie zeigen *keinen* statistisch nachweisbaren Zusammenhang mit Gate-Anzahl, Klauselzahl, Eingangszahl oder Tiefe ($|\rho| \leq 0{,}11$, alle $p > 0{,}1$) und unterscheiden sich weder zwischen den Circuit-Typen noch zwischen den Suites signifikant. Rund 60 % der Messwerte liegen innerhalb von ± 1 ms um das Einfache oder Doppelte von 15,625 ms, der typischen Auflösung der verwendeten Wallclock-Messung. Wir schließen daraus, dass die Messungen durch Prozessstart und Zeitauflösung dominiert werden und nur eine *obere Schranke* der Solver-Laufzeit liefern, jedoch keine Aussage über das Skalierungsverhalten des Algorithmus erlauben. Zusätzlich dokumentieren wir, dass in allen Läufen weder eine Solver-Ausgabe noch ein Erfüllbarkeitsergebnis erfasst wurde und dass der CNF-Encoder die Gate-Typen NAND, NOR und XOR fehlerhaft kodiert. Wir leiten daraus konkrete Verbesserungen des Messaufbaus ab.

Inhaltsverzeichnis

1 Einführung **4**
- 1.1 Fragestellungen . . . 4
- 1.2 Beitrag . . . 4

2 Grundlagen **4**
- 2.1 Boolean Circuits und CNF . . . 4
- 2.2 Implementierung des Subgraph-SAT-Solvers . . . 5
  - 2.2.1 Einlesen und Normalisieren der Formel . . . 5
  - 2.2.2 Reduktion von 3-SAT auf Clique . . . 5
  - 2.2.3 Reduktion von Clique auf Subgraph-Isomorphismus . . . 7
  - 2.2.4 Ergebnisverarbeitung und Belegung . . . 7
  - 2.2.5 Laufzeitmessung und Einordnung . . . 7

3 Experimenteller Aufbau **8**
3.1 Circuit-Familien . . . 8
3.1.1 Target-AND . . . 8
3.1.2 Target-AND-Random . . . 8
3.1.3 ISCAS'85 . . . 9
3.2 CNF-Kodierung . . . 9
3.3 Messmethodik . . . 9
3.4 Experiment-Suites . . . 10
3.5 Statistische Auswertung . . . 10

4 Ergebnisse **10**
4.1 Gesamtüberblick . . . 10
4.2 Target-AND: Variation der Eingangszahl (F1) . . . 12
4.3 Target-AND-Random: Variation der Tiefe (F2) . . . 13
4.4 ISCAS'85-Benchmarks (F3) . . . 15
4.5 Zusammenhang zwischen Instanzgröße und Laufzeit . . . 17
4.6 Vergleich der Circuit-Typen und der Suites (F4) . . . 18
4.7 Zeitliche Auflösung der Messung . . . 21
4.8 Solver-Ausgabe und Erfüllbarkeit . . . 21

5 Diskussion **21**
5.1 Was die Daten stützen . . . 21
5.2 Was die Daten nicht stützen . . . 22
5.3 Gültigkeitsbedrohungen . . . 22
5.4 Verbesserungen des Messaufbaus . . . 23

6 Neue Version der Experimente (Version 2.0.0) **23**
6.1 Einordnung und Abgrenzung zur ersten Messreihe . . . 24
6.2 Hardware- und Softwareumgebung . . . 24
6.3 Aufbau der neuen Experimente . . . 26
6.3.1 Messablauf . . . 26
6.3.2 Die beiden Solver . . . 27
6.3.3 Instanzen . . . 27
6.3.4 Datenbasis dieses Abschnitts . . . 28
6.4 Ergebnisse der neuen Experimente . . . 29
6.4.1 Basislinie: Prozessstart-Grundlast . . . 29
6.4.2 Gesamtüberblick . . . 30
6.4.3 Experiment 1: Target-AND und Skalierung . . . 32
6.4.4 Experiment 2: Target-AND-Random, Variation der Tiefe . . . 34
6.4.5 Experiment 3: ISCAS'85 . . . 35
6.4.6 Miter-Instanzen: unerfüllbare Formeln . . . 38
6.4.7 Vergleich der beiden Solver . . . 39
6.4.8 Korrektheit und Übereinstimmung der Solver . . . 40
6.5 Diskussion der neuen Version . . . 41
6.5.1 Was die neuen Daten stützen . . . 41
6.5.2 Was die neuen Daten nicht stützen . . . 41
6.5.3 Gültigkeitsbedrohungen . . . 42
6.5.4 Vergleich der Befunde mit der ersten Messreihe . . . 42
6.5.5 Offene Punkte . . . 43

6.6 Reproduktion der neuen Auswertung 43

7 Zusammenfassung **44**
7.1 Ausblick 44
7.2 Implementierungshinweis 44

A Reproduzierbarkeit und Datenquellen **44**

B Beispiel-Datensätze **45**

C Diagramme der einzelnen Suites **45**

1 Einführung

Der Subgraph Algorithmus [1] vergleicht zwei Graphen anhand von Adjazenzmatrizen und Signatur-Arrays und besitzt eine Laufzeit von $O(n^3)$. Der Subgraph-SAT-Solver überträgt diese Idee auf das Erfüllbarkeitsproblem aussagenlogischer Formeln in konjunktiver Normalform (CNF). Die hier verwendeten Target-AND-Circuits und die zugehörigen Experimentreihen gehen auf die Masterarbeit des Autors [2] zurück. Das vorliegende Projekt `subgraph-sat-solver-experiments` ist ein Python-Framework, das Boolean Circuits erzeugt, sie in das DIMACS-CNF-Format übersetzt, den Solver als externen Prozess aufruft, die Laufzeit misst und die Ergebnisse als JSON, CSV und Diagramme ablegt.

Diese Arbeit führt alle bisher erzeugten Ergebnisse dieses Frameworks zusammen und wertet sie statistisch aus.

1.1 Fragestellungen

Die Experimente sollten folgende Fragen beantworten:

F1 Wie verhält sich die Laufzeit des Solvers bei wachsender Anzahl der Eingänge m (Target-AND-Circuits)?

F2 Wie verhält sich die Laufzeit bei wachsender Tiefe d zufälliger Zwischenschichten (Target-AND-Random)?

F3 Wie verhält sich der Solver auf industriellen Standard-Benchmarks (ISCAS'85)?

F4 Wie reproduzierbar sind die Messungen über verschiedene Suites hinweg?

1.2 Beitrag

- Vollständige Darstellung des Experimentaufbaus und aller 288 Messwerte (Abschnitte 3 und 4).
- Eine statistische Auswertung (Korrelationen, Rangtests), die zeigt, dass die Laufzeit in den vorliegenden Messungen unabhängig von der Instanzgröße ist.
- Eine kritische Einordnung der Messmethodik: Zeitauflösung, fehlende Solver-Ausgabe und Kodierung (Abschnitt 5).
- Ein reproduzierbares Auswertungsskript (`src/paper_analysis.py`), das alle Tabellen und Abbildungen dieser Arbeit direkt aus den Rohdaten erzeugt (Anhang A).

2 Grundlagen

2.1 Boolean Circuits und CNF

Definition 1 (Boolean Circuit)**.** Ein Boolean Circuit mit m Eingängen ist eine geordnete Liste von Gates $g_1, \ldots, g_k$. Jedes Gate hat einen Typ aus $\{\text{NOT}, \text{AND}, \text{OR}, \text{NAND}, \text{NOR}, \text{XOR}\}$ und bezieht seine Eingaben von Eingängen oder von früheren Gates. Die Größe des Circuits ist die Gate-Anzahl k.

Definition 2 (CNF)**.** Eine Formel in konjunktiver Normalform (CNF) ist eine Konjunktion von Klauseln, wobei jede Klausel eine Disjunktion von Literalen ist. Das Erfüllbarkeitsproblem (SAT) fragt, ob eine Belegung der Variablen existiert, die alle Klauseln erfüllt [5].

Die Übersetzung eines Circuits in CNF erfolgt mit der Tseitin-Transformation [4]: Jedes Gate erhält eine eigene Variable y, und lokale Klauseln erzwingen $y \leftrightarrow \mathrm{op}(x_1, \ldots)$. Die Formelgröße wächst dadurch linear in der Gate-Anzahl.

2.2 Implementierung des Subgraph-SAT-Solvers

Der untersuchte Solver ist ein in C++17 implementiertes Kommandozeilenprogramm und zugleich als statische Bibliothek nutzbar. Sein SAT-Verfahren besteht aus einer Reduktionspipeline und einem separaten Subgraph-Programm; der C++-Teil implementiert also nicht selbst den Subgraph Algorithmus. Abbildung und Phasenmessung lassen sich wie folgt zusammenfassen:

`DIMACS-CNF` $\rightarrow$ `3-SAT` $\rightarrow$ `Clique-Graph` $\rightarrow$ `Subgraph-Instanz` $\rightarrow$ `externes Programm`

2.2.1 Einlesen und Normalisieren der Formel

Die Kommandozeilenschnittstelle erwartet eine DIMACS-CNF-Datei. Alternativ kann die Bibliothek eine Formel als Zeichenkette oder als bereits geparstes Objekt entgegennehmen. Literale werden intern als vorzeichenbehaftete ganze Zahlen dargestellt: Eine positive Zahl bezeichnet eine Variable, eine negative deren Negation. Vor der Graphkonstruktion kopiert der Reduktor die Formel und wandelt Klauseln mit mehr als drei Literalen durch zusätzliche Hilfsvariablen in Klauseln mit höchstens drei Literalen um. Klauseln mit höchstens drei Literalen bleiben unverändert. Anschließend entfernt `simplify()` doppelte Klauseln und Tautologien; die separat vorhandene Unit-Propagation-Routine wird in diesem Ausführungspfad nicht aufgerufen. Die Reduktion verändert damit eine interne Kopie, während die ursprüngliche Formel für die spätere Ergebnisvalidierung erhalten bleibt.

2.2.2 Reduktion von 3-SAT auf Clique

Für jedes Literalvorkommen wird ein eigener Knoten angelegt. Ein Knoten speichert dabei implizit, aus welcher Klausel und an welcher Position das Literal steht. Zwei Knoten werden genau dann durch eine ungerichtete Kante verbunden, wenn sie aus verschiedenen Klauseln stammen und ihre Literale nicht unmittelbar im Widerspruch stehen. Für die Literale x_i und x_j bedeutet das insbesondere, dass zwischen einem Vorkommen von x und einem Vorkommen von $\neg x$ keine Kante liegt. Zwischen zwei Knoten derselben Klausel gibt es ebenfalls keine Kante.

Gesucht wird eine Clique der Größe k, wobei k der Anzahl der Klauseln der normalisierten Formel entspricht. Eine erfüllende Belegung liefert eine solche Clique, indem aus jeder Klausel ein unter dieser Belegung wahres Literal ausgewählt wird: Die ausgewählten Knoten stammen aus verschiedenen Klauseln und widersprechen einander nicht. Umgekehrt enthält eine Clique der Größe k wegen der fehlenden Kanten innerhalb einer Klausel genau einen Knoten je Klausel; wegen der fehlenden Kanten zwischen komplementären Literalen sind die gewählten Literale miteinander verträglich. Sie können daher gemeinsam

wahr gesetzt werden und erfüllen jede Klausel. Dies ist die intendierte Korrektheitsbeziehung der Reduktion.

Der Clique-Graph wird durch eine boolesche Adjazenzmatrix repräsentiert. Bezeichnet L die Zahl der Literalvorkommen nach der Normalisierung, besitzt der Graph L Knoten; die Kanten werden durch paarweisen Vergleich der Knoten konstruiert. Die Konstruktion benötigt damit $O(L^2)$ Paarprüfungen und die Matrix $O(L^2)$ Speicherplätze. Der Code prüft vor dieser Stufe, dass keine Klausel mehr als drei Literale enthält.

Listing 1 zeigt den Kern dieser Konstruktion in C++17. Für jedes Literalvorkommen wird ein Knoten angelegt; die Adjazenzmatrix erhält genau dann eine Kante, wenn zwei Knoten aus verschiedenen Klauseln stammen und ihre Literale nicht komplementär sind. Der Rückgabewert enthält außerdem die gesuchte Clique-Größe k, die der Anzahl der Klauseln entspricht. Das Beispiel erläutert die Reduktionsidee und ist kein wörtlicher Auszug aus dem Solver-Quelltext.

Listing 1: Didaktischer C++17-Auszug: Konstruktion der Clique-Instanz aus einer 3-CNF-Formel

```
#include <cstddef>
#include <stdexcept>
#include <utility>
#include <vector>

struct CliqueInstance {
    std::vector<std::vector<bool>> adjacency;
    std::size_t targetSize;
};

struct LiteralNode {
    std::size_t clause;
    int literal;
};

CliqueInstance makeCliqueInstance(
    const std::vector<std::vector<int>>& clauses) {
    std::vector<LiteralNode> nodes;
    for (std::size_t c = 0; c < clauses.size(); ++c) {
        if (clauses[c].size() > 3) {
            throw std::invalid_argument("formula is not 3-CNF");
        }
        for (int literal : clauses[c]) {
            nodes.push_back({c, literal});
        }
    }

    std::vector<std::vector<bool>> adjacency(
        nodes.size(), std::vector<bool>(nodes.size(), false));
    for (std::size_t i = 0; i < nodes.size(); ++i) {
        for (std::size_t j = i + 1; j < nodes.size(); ++j) {
            const auto& a = nodes[i];
            const auto& b = nodes[j];
            const bool differentClauses = a.clause != b.clause;
            const bool complementary = a.literal == -b.literal;
```

```
36                if (differentClauses && !complementary) {
37                    adjacency[i][j] = true;
38                    adjacency[j][i] = true;
39                }
40            }
41        }
42        return {std::move(adjacency), clauses.size()};
43    }
```

2.2.3 Reduktion von Clique auf Subgraph-Isomorphismus

Der Clique-Graph wird zum Quellgraphen G. Als Zielgraph H wird der vollständige Graph K_k mit k Knoten erzeugt. Da jede Kante von K_k im Quellgraphen vorhanden sein muss, ist K_k genau dann ein Teilgraph von G, wenn G eine Clique der Größe k enthält. Beide Graphen werden mit Knotenbezeichnern wie `v0` und einer Kantenliste in JSON serialisiert. Die äußere JSON-Struktur enthält die Schlüssel `G` und `H`.

Der Solver schreibt diese Daten in temporäre Dateien und startet das konfigurierte Subgraph-Binary als separaten Prozess. Unter Windows geschieht dies über `system()`, unter Unix-artigen Systemen über `fork()` und `execl()`. Das externe Programm muss die Eingabe- und Ausgabedatei als Kommandozeilenargumente akzeptieren. Damit hängt die praktische Entscheidung nicht nur von der Reduktionsimplementierung, sondern auch von einem kompatiblen externen Binary samt JSON-Schnittstelle ab.

2.2.4 Ergebnisverarbeitung und Belegung

Die Ausgabedatei des Subgraph-Programms wird nicht als JSON geparst. Stattdessen sucht der Solver nach der exakten Zeichenfolge `is_subgraph: true`. Wird sie gefunden, gilt die Subgraph-Instanz als positiv; andernfalls wird sie als negativ und damit die SAT-Formel als unerfüllbar eingestuft. Ein Prozessfehler führt zu `SAT_ERROR`. Die Konfiguration enthält zwar ein Zeitlimit, der Prozessaufruf setzt dieses in der vorliegenden Implementierung jedoch nicht durch.

Für eine vollständige SAT-Lösung müsste das vom Subgraph-Programm gefundene Knotenmapping zurückgegeben und auf die ausgewählten Literalvorkommen abgebildet werden. Dieser Schritt fehlt im aktuellen Aufrufpfad: Das Mapping wird weder aus der Ausgabe gelesen noch an den vorhandenen Extraktor übergeben. Bei positivem Subgraph-Ergebnis setzt der Solver stattdessen jede Variable der ursprünglichen Formel auf wahr. Die standardmäßig aktive Validierung prüft diese Belegung gegen die ursprüngliche CNF; schlägt sie fehl, lautet das Ergebnis `SAT_ERROR`. Ein positives Subgraph-Ergebnis allein garantiert daher in dieser Implementierung noch keine zurückgegebene, gültige SAT-Belegung.

2.2.5 Laufzeitmessung und Einordnung

Der Ergebnistyp erfasst getrennte Zeiten für Reduktion, Aufruf des Subgraph-Prozesses und Extraktion sowie eine Gesamtzeit. Die Zeit des Prozessaufrufs umfasst dabei auch die zugehörige Datei-I/O und den Prozessstart; sie ist keine isolierte Messung der Rechenzeit des externen Algorithmus. Die Reduktionsstufe baut zunächst einen Graphen auf, dessen Größe quadratisch mit der Zahl der Literalvorkommen wachsen kann. Die in der

theoretischen Grundlage angegebene Schranke $\Theta(n^3)$ bezieht sich auf den Subgraph Algorithmus [1]; sie beschreibt weder automatisch die Kosten der gesamten Implementierung noch belegt sie deren Korrektheit. Ob und wie sich diese Schranke im gemessenen Prozess auswirkt, ist nicht Gegenstand der Experimente (Abschnitt 5).

Im Experiment-Framework wird das kompilierte Programm als Black Box aufgerufen: Das Framework erzeugt die Eingabe-CNF, startet den Prozess und misst die Wallclock-Zeit bis zu dessen Ende (Abschnitt 3.3). Diese Messung ist von den internen Phasenzeiten der Solver-Bibliothek zu unterscheiden. Außerdem dokumentieren die Experimente, dass bei keinem der 288 Läufe ein SAT-Ergebnis erfasst wurde; somit weisen die Messdaten nicht nach, dass die beschriebene Pipeline in diesen Läufen eine gültige Lösung geliefert hat.

3 Experimenteller Aufbau

3.1 Circuit-Familien

3.1.1 Target-AND

Ein Target-AND-Circuit mit m Eingängen $x_0, \ldots, x_{m-1}$ ist so konstruiert, dass sein Ausgang nur unter genau einer Eingabebelegung a^* wahr wird. Diese Belegung wird durch die Halbierungsstrategie des LBV-Verfahrens (vgl. [2]) bestimmt: Ausgehend von der Nullbelegung werden in jeder Runde die ersten $\lfloor m/2^r \rfloor$ Eingänge invertiert, $r = 1, 2, \ldots$, bis die Blockgröße 0 erreicht. Der Circuit besteht aus je einem NOT-Gate pro Eingang und einem balancierten Binärbaum aus AND-Gates über die Literale von a^*.

Satz 1 (Größe der Target-AND-Circuits)**.** Ein Target-AND-Circuit mit $m \geq 2$ Eingängen besitzt genau $2m - 1$ Gates und seine Tseitin-CNF genau $6m - 4$ Klauseln.

Beweis. Es gibt m NOT-Gates. Ein Binärbaum mit m Blättern hat unabhängig von m genau $m - 1$ innere Knoten; jeder ist ein AND-Gate. Das ergibt $m + (m - 1) = 2m - 1$ Gates. Ein NOT-Gate erzeugt zwei Klauseln, ein AND-Gate mit zwei Eingängen vier (siehe Abschnitt 3.2). Insgesamt ergeben sich $2m + 4(m - 1) = 6m - 4$ Klauseln. □

Die Messdaten bestätigen dies für alle $m \in \{4, 6, \ldots, 20\}$ (Tabelle 4); beispielsweise besitzt $m = 20$ genau 39 Gates und 116 Klauseln. Die Anzahl der LBV-Runden, die im Datensatz als Feld `lbv_rounds` gespeichert ist, beträgt $\lfloor \log_2 m \rfloor + 1$ (3 Runden für $m \in \{4, 6\}$, 4 für $m \in \{8, \ldots, 14\}$, 5 für $m \in \{16, 18, 20\}$).

3.1.2 Target-AND-Random

Dieser Typ erweitert den Target-AND-Circuit mit $m = 10$ Eingängen um $d \in \{1, \ldots, 12\}$ zufällige Zwischenschichten. Jede Schicht enthält $\max(2, \lfloor |L|/2 \rfloor + r)$ Gates mit $r \in \{0, 1, 2\}$ gleichverteilt, wobei $|L|$ die Größe der vorherigen Schicht ist. Jedes Gate hat einen zufälligen Typ aus $\{\text{AND}, \text{OR}, \text{NAND}, \text{NOR}, \text{XOR}\}$ und zwei oder drei zufällig gewählte Eingänge der vorherigen Schicht. Ein abschließendes AND-Gate verknüpft das Target-AND mit dem ersten Gate der letzten Zufallsschicht. Der Zufallsgenerator wird in jedem Circuit mit dem festen Seed 42 initialisiert, sodass die Circuits deterministisch sind und der Circuit der Tiefe $d + 1$ den Circuit der Tiefe d (bis auf das abschließende AND-Gate) um eine weitere Schicht ergänzt.

3.1.3 ISCAS'85

Die ISCAS'85-Benchmarks [3] sind kombinatorische Standard-Schaltkreise aus der Hardware-Verifikation. Das Framework liest die `.bench`-Dateien aus `src/iscas85/` ein. Es werden die zehn klassischen Circuits c432 bis c7552 sowie zwei kleine Beispielcircuits (c3 mit 4 und c17 mit 6 Gates) verwendet. Zwei Circuits sind funktional verwandt: c1355 ist die Variante von c499, in der die XOR-Gates durch NAND-Gates ersetzt wurden.

3.2 CNF-Kodierung

Für ein Gate mit Variable y und Eingangsvariablen $x_1, \ldots, x_k$ erzeugt das Framework folgende Klauseln:

Gate	Klauseln	Anzahl
NOT	$(\neg y \vee \neg x_1),\ (y \vee x_1)$	2
AND	$(\neg y \vee x_1 \vee \ldots \vee x_k),\ (\neg y \vee x_i)_i,\ (y \vee \neg x_1 \vee \ldots \vee \neg x_k)$	$k+2$
OR	$(y \vee \neg x_1 \vee \ldots \vee \neg x_k),\ (y \vee \neg x_i)_i,\ (\neg y \vee x_1 \vee \ldots \vee x_k)$	$k+2$
NAND, NOR	Klauselmuster von OR bzw. AND (siehe unten)	$k+2$
XOR	vier Klauseln über y, x_1, x_2 (siehe unten)	4

Eine Überprüfung der Kodierung für jedes Gate-Muster mit allen Eingabekombinationen ergibt, dass AND, OR und NOT korrekt kodiert sind, NAND, NOR und XOR jedoch nicht: Im Quelltext verwendet NAND dasselbe Klauselmuster wie OR, NOR dasselbe wie AND, und die vier XOR-Klauseln beschreiben $y \leftrightarrow \neg(x_1 \oplus x_2)$ (XNOR). Damit beschreiben die erzeugten CNF-Dateien für Gates dieser drei Typen nicht die Funktion des ursprünglichen Circuits. Das betrifft vor allem ISCAS'85, dessen Circuits überwiegend aus NAND-, NOR- und XOR-Gates bestehen (c17 enthält ausschließlich NAND-Gates, c6288 besteht zu etwa 88 % aus NAND-, NOR- und XOR-Gates), sowie die zufälligen Zwischenschichten von Target-AND-Random. Die Target-AND-Circuits verwenden nur NOT und AND und sind nicht betroffen. Wir haben die Kodierung für diese Arbeit nicht verändert, damit die dokumentierten Ergebnisse den tatsächlich ausgeführten Experimenten entsprechen.

Die Kodierung enthält *keine* Unit-Klausel, die den Ausgang des Circuits auf wahr festlegt. Jede Eingabebelegung lässt sich daher zu einer erfüllenden Belegung der Gesamtformel fortsetzen, die erzeugten Formeln sind unabhängig vom Circuit erfüllbar. Auf diesen Punkt kommen wir in Abschnitt 5 zurück.

3.3 Messmethodik

Jeder Lauf schreibt die CNF in eine temporäre Datei und startet den Solver per `subprocess.run` mit einem Timeout von 30 s. Gemessen wird die Wallclock-Zeit t zwischen `time.time()` vor dem Start und nach dem Ende des Prozesses, in Millisekunden. Die gemessene Zeit umfasst damit Prozessstart, Einlesen der Datei, Lösen und Prozessende. Als erfüllbar gilt eine Instanz, wenn die Zeichenkette `SAT` in Standardausgabe oder Fehlerausgabe vorkommt und `UNSAT` nicht. Der Speicherverbrauch wird nicht erfasst (das Feld `memory_usage_mb` ist stets 0). Pro Lauf werden Circuit-Typ, Name, Anzahl Eingänge, Gates und Klauseln, Erfüllbarkeit, Laufzeit, Zeitstempel, LBV-Runden und Metadaten als JSON und CSV gespeichert.

3.4 Experiment-Suites

Die vier Suites unterscheiden sich in Umfang und Wiederholungszahl (Tabelle 1). Quick und Standard enthalten jeden Circuit einmal, Full und Performance fünfmal (Target-AND und Target-AND-Random) bzw. einmal (ISCAS'85). Insgesamt entstehen 288 Läufe: 108 Target-AND, 144 Target-AND-Random und 36 ISCAS'85. Full und Performance sind in Umfang und Zusammensetzung identisch. Die Suites wurden am 14. Juli 2026 zwischen 09:40:03 und 09:40:48 Uhr ausgeführt.

Suite	n	T-AND	T-AND-R	ISCAS	Dauer (s)	$\bar{t}$ (ms)	s (ms)	$\tilde{t}$ (ms)
Quick	21	9	12	0	≤ 2	24,52	7,65	27,57
Standard	33	9	12	12	≤ 2	22,46	7,06	22,56
Full	117	45	60	12	≤ 4	21,68	7,06	19,84
Performance	117	45	60	12	≤ 3	21,27	7,39	18,00
Gesamt	288	108	144	36	–	21,81	7,25	19,65

Tabelle 1: Die vier Experiment-Suites. T-AND: Target-AND, T-AND-R: Target-AND-Random. $\bar{t}$: Mittelwert, s: Standardabweichung, $\tilde{t}$: Median der Laufzeit. Die Dauer ist aus den Zeitstempeln (Auflösung 1 s) als obere Schranke abgeleitet.

3.5 Statistische Auswertung

Alle Kennzahlen werden aus den CSV-Dateien der Suites berechnet. Verwendet werden Mittelwert $\bar{t}$, Standardabweichung s (mit $n-1$), Median, Quartile Q_1, Q_3, Minimum und Maximum. Zusammenhänge werden mit der Rangkorrelation nach Spearman (ρ) und der Pearson-Korrelation (r) sowie einer linearen Regression geprüft. Da die Laufzeiten deutlich nicht normalverteilt sind (Abbildung 1), verwenden wir für Gruppenvergleiche rangbasierte Tests: Kruskal–Wallis für mehr als zwei Gruppen und Mann–Whitney-U für zwei Gruppen. Das Signifikanzniveau beträgt $\alpha = 0{,}05$.

4 Ergebnisse

4.1 Gesamtüberblick

Über alle 288 Läufe liegt die Laufzeit zwischen 6,02 ms und 40,27 ms; der Mittelwert beträgt 21,81 ms, die Standardabweichung 7,25 ms und der Median 19,65 ms. Kein Lauf endete mit einem Fehler oder einem Timeout. Die Instanzen decken eine große Größenspanne ab (Tabelle 2): von 4 bis 3 512 Gates (Faktor 878) und von 14 bis 10 690 Klauseln (Faktor 764).

Tabelle 3 zeigt die Laufzeitstatistik für jede Kombination aus Suite und Circuit-Typ. Die Mittelwerte liegen in allen elf Zellen zwischen 19,2 und 25,7 ms, die Mediane zwischen 16,0 und 28,3 ms.

Aufbau der Abbildung. Abbildung 1 ist ein gestapeltes Histogramm. Auf der Abszisse ist die gemessene Laufzeit in Millisekunden aufgetragen, unterteilt in Klassen der Breite 1 ms (von 5 bis 41 ms); die Ordinate zählt die Läufe je Klasse. Jede Säule besteht aus bis zu

Circuit-Typ	Instanzen	Eingänge	Gates	$\bar{g}$	Klauseln	$\bar{k}$
Target-AND	9	4–20	7–39	23,0	20–116	68,0
Target-AND-Random	12	10	27–54	40,9	91–206	151,2
ISCAS'85	12	4–233	4–3 512	1106,5	14–10 690	3644,7

Tabelle 2: Strukturelle Eigenschaften der Instanzen je Circuit-Typ (pro eindeutigem Circuit). $\bar{g}$: mittlere Gate-Anzahl, $\bar{k}$: mittlere Klauselzahl.

Suite	Circuit-Typ	n	$\bar{t}$	s	Min	Q_1	Median	Q_3	Max
Quick	Target-AND	9	22,92	8,57	15,75	15,77	16,00	31,40	35,50
Quick	Target-AND-Random	12	25,72	7,02	15,95	17,68	28,32	31,77	31,98
Standard	Target-AND	9	23,91	4,30	15,89	22,56	24,41	25,55	30,96
Standard	Target-AND-Random	12	21,76	7,95	10,97	15,96	19,92	28,85	33,70
Standard	ISCAS'85	12	22,08	8,12	13,93	15,81	17,70	31,54	35,24
Full	Target-AND	45	21,10	7,35	9,60	15,78	16,11	31,60	32,80
Full	Target-AND-Random	60	22,61	7,02	7,91	15,94	22,87	28,68	40,27
Full	ISCAS'85	12	19,23	5,71	11,54	15,74	16,03	22,76	31,87
Performance	Target-AND	45	21,69	8,16	7,62	15,90	16,20	31,74	33,83
Performance	Target-AND-Random	60	21,34	7,04	6,02	15,84	19,83	27,12	32,05
Performance	ISCAS'85	12	19,38	6,27	15,69	15,88	16,06	19,25	32,06

Tabelle 3: Laufzeitstatistik (in ms) je Suite und Circuit-Typ. Alle Werte stammen aus den `experiment_results.csv`-Dateien der Suites.

drei übereinandergestapelten Farbsegmenten: Target-AND (violett, unten), Target-AND-Random (orange, Mitte) und ISCAS'85 (blau, oben). Die Gesamthöhe einer Säule ist die Anzahl aller Läufe in dieser Klasse, die Höhe eines Segments die Anzahl der Läufe des jeweiligen Typs. Die beiden roten, gestrichelten senkrechten Linien markieren 15,625 ms (= 1/64 s) und 31,25 ms (= 2/64 s); ihre Bedeutung wird in Abschnitt 4.7 erläutert. Da die drei Typen mit 108, 144 und 36 Läufen unterschiedlich stark vertreten sind, lassen sich die Segmenthöhen nicht unmittelbar als Anteile je Typ lesen.

Die beiden dominierenden Säulen. Das Bild wird von zwei hohen, schmalen Säulen beherrscht, die jeweils unmittelbar auf einer der roten Linien stehen. Die Klasse $[15; 16)$ ms enthält 76 Läufe (26,4 % aller Läufe), nämlich 33 Target-AND-, 28 Target-AND-Random- und 15 ISCAS-Läufe. Die Klasse $[31; 32)$ ms enthält 60 Läufe (20,8 %), davon 26, 31 bzw. 3. Diese beiden von insgesamt 36 Klassen vereinen damit 136 der 288 Läufe (47,2 %). Rechts neben der ersten Säule schließt sich als Schulter die Klasse $[16; 17)$ ms mit 30 Läufen an (11 Target-AND, 15 Target-AND-Random, 4 ISCAS), rechts neben der zweiten Säule die Klasse $[32; 33)$ ms mit 8 Läufen. In beiden Säulen sind alle drei Farben vertreten; die Häufungen sind also nicht an einen einzelnen Circuit-Typ gebunden. Auffällig ist lediglich, dass ISCAS'85 mit 15 von 36 Läufen (41,7 %) in der unteren, aber nur mit 3 von 36 Läufen (8,3 %) in der oberen Säule steht. Bei 36 Läufen und dem nicht signifikanten Mann–Whitney-Test zwischen ISCAS'85 und den synthetischen Circuits ($p = 0{,}140$, Tabelle 9) lässt sich daraus jedoch keine Tendenz ableiten.

Der Bereich dazwischen und die Ränder. Zwischen den beiden Säulen verläuft ein flacher Bereich: Die 14 Klassen von 17 bis 31 ms enthalten zusammen 86 Läufe, im Mittel gut 6 je Klasse. Die höchste dieser Klassen ist $[23; 24)$ ms mit 13 Läufen; zu größeren

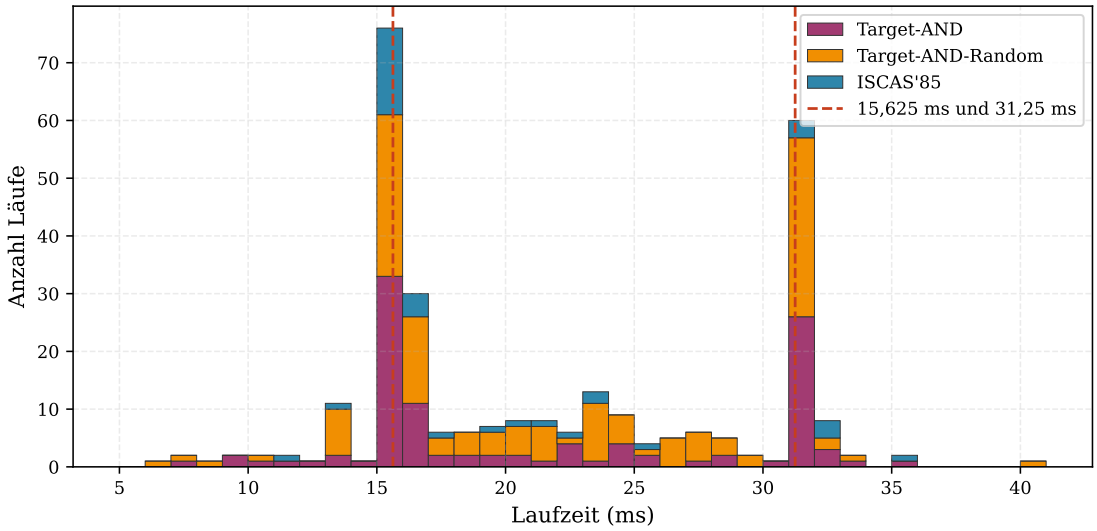


Abbildung 1: Verteilung aller 288 Laufzeiten (Klassenbreite 1 ms), gestapelt nach Circuit-Typ: Target-AND (violett), Target-AND-Random (orange), ISCAS'85 (blau). Die roten gestrichelten Linien markieren 15,625 ms und 31,25 ms.

Laufzeiten hin wird der Bereich dünner, bis die Klasse $[30; 31)$ ms nur noch einen einzigen Lauf enthält. In diesem Mittelbereich überwiegen die orangen Segmente: 54 der 86 Läufe stammen von Target-AND-Random, 24 von Target-AND und 8 von ISCAS'85. Am linken Rand liegen 23 Läufe unterhalb von 15 ms, verteilt auf die Klassen von 6 bis 14 ms. Dort fällt die Klasse $[13; 14)$ ms mit 11 Läufen (8 Target-AND-Random, 2 Target-AND, 1 ISCAS) als kleine Nebenspitze auf; ihre Ursache lässt sich aus den Daten nicht bestimmen. Am rechten Rand liegen nur 5 Läufe oberhalb von 33 ms (33,70, 33,83, 35,24, 35,50 und 40,27 ms). Der Maximalwert von 40,27 ms ist ein einzelner Ausreißer (Target-AND-Random, $d = 2$) in der sonst leeren Klasse $[40; 41)$ ms.

Interpretation. Dass fast die Hälfte aller Läufe in zwei Klassen liegt, deren Mitten genau um einen Zeittakt von 15,625 ms auseinanderliegen, ist der zentrale Befund dieser Abbildung. Wäre die gemessene Zeit im Wesentlichen die Rechenzeit des Solvers, wäre für Instanzen von 4 bis 3 512 Gates eine breite, mit der Größe verschobene Verteilung zu erwarten. Die beobachtete Form ist dagegen mit einer Messung vereinbar, deren Zeitauflösung grob ist (Abschnitt 4.7). Die Verteilung sollte daher nicht als Mischung aus „schnellen“ und „langsamen“ Instanzen gedeutet werden.

4.2 Target-AND: Variation der Eingangszahl (F1)

Tabelle 4 fasst die Läufe nach Eingangszahl m zusammen (je 12 Läufe über alle Suites). Gate- und Klauselzahl wachsen wie in Satz 1 linear, von 7 auf 39 Gates. Die mittlere Laufzeit liegt für alle m zwischen 17,49 ms ($m = 4$) und 24,29 ms ($m = 20$) und zeigt keinen Trend (Abbildung 2, links). Die Streuung innerhalb eines m-Wertes ($s = 6$ bis 8,6 ms) ist größer als der Unterschied der Mittelwerte zwischen verschiedenen m. Die Rangkorrelation zwischen Gate-Anzahl und Laufzeit beträgt $\rho = 0{,}106$ ($p = 0{,}275$, $n = 108$).

m	Gates	Klauseln	n	$\bar{t}$	s	Min	Median	Max
4	7	20	12	17,49	6,14	7,62	15,97	31,76
6	11	32	12	24,23	7,28	15,68	23,91	31,95
8	15	44	12	21,25	6,02	15,67	19,84	32,80
10	19	56	12	19,09	7,88	9,56	16,05	35,50
12	23	68	12	22,04	8,01	13,25	16,17	31,98
14	27	80	12	21,94	7,70	15,74	16,05	31,92
16	31	92	12	23,61	8,13	11,49	22,55	33,83
18	35	104	12	21,64	8,56	9,60	17,04	32,26
20	39	116	12	24,29	7,65	15,67	26,78	31,83

Tabelle 4: Target-AND-Circuits: Struktur und Laufzeit (ms) je Eingangszahl m, über alle vier Suites aggregiert.

4.3 Target-AND-Random: Variation der Tiefe (F2)

Bei festem $m = 10$ wächst die Instanzgröße mit der Tiefe d von 27 Gates und 91 Klauseln ($d = 1$) auf 54 Gates und 206 Klauseln ($d = 12$) (Tabelle 5). Die mittlere Laufzeit schwankt unsystematisch zwischen 18,79 ms ($d = 8$) und 26,57 ms ($d = 5$); sie ist bei der größten Tiefe $d = 12$ mit 20,56 ms kleiner als bei $d = 1$ mit 23,08 ms. Die Rangkorrelation zwischen Tiefe und Laufzeit ist $\rho = -0{,}086$ ($p = 0{,}306$, $n = 144$). Die lineare Regression liefert eine Steigung von $-0{,}28$ ms pro Tiefenschicht mit $R^2 = 0{,}018$. Ein tiefenabhängiger Anstieg ist damit nicht erkennbar (Abbildung 2, rechts).

d	Gates	Klauseln	n	$\bar{t}$	s	Min	Median	Max
1	27	91	12	23,08	8,16	10,97	22,29	31,87
2	30	104	12	22,70	8,69	13,81	18,60	40,27
3	33	117	12	22,86	7,12	13,88	23,01	31,82
4	35	127	12	23,48	7,55	15,74	20,78	32,04
5	38	139	12	26,57	5,33	15,75	27,43	32,05
6	40	149	12	20,77	5,70	15,71	17,97	31,78
7	42	157	12	22,76	6,06	15,81	22,40	31,89
8	44	165	12	18,79	4,74	13,20	17,01	28,66
9	47	177	12	23,85	6,28	13,33	25,20	31,86
10	49	185	12	21,99	8,52	6,02	20,41	33,70
11	52	197	12	19,85	6,42	13,41	17,19	31,94
12	54	206	12	20,56	9,30	7,91	17,25	31,96

Tabelle 5: Target-AND-Random-Circuits ($m = 10$): Struktur und Laufzeit (ms) je Tiefe d, über alle vier Suites aggregiert.

Aufbau der Abbildung. Abbildung 2 besteht aus zwei nebeneinanderliegenden Teilbildern mit gemeinsamer Ordinate (Laufzeit in ms, etwa von 5 bis 42 ms). Im linken Teilbild ist die Eingangszahl $m = 4, 6, \ldots, 20$ der Target-AND-Circuits auf der Abszisse aufgetragen (neun Positionen, entsprechend 7 bis 39 Gates), im rechten die Tiefe $d = 1, \ldots, 12$ der Target-AND-Random-Circuits bei festem $m = 10$ (zwölf Positionen, entsprechend 27 bis 54 Gates). An jeder Position sind alle 12 Einzelläufe als halbtransparente graue Punkte eingezeichnet; wo mehrere Läufe nahezu denselben Wert haben, überlagern sich die

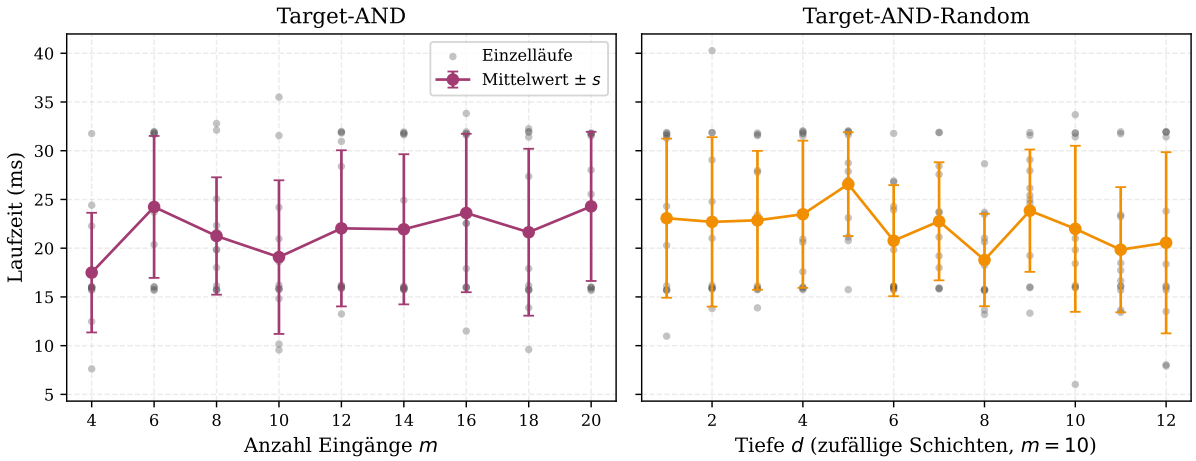


Abbildung 2: Laufzeit in Abhängigkeit von der Eingangszahl m (links, Target-AND) und der Tiefe d (rechts, Target-AND-Random mit $m = 10$); beide Teilbilder teilen sich die Ordinate. Graue Punkte: Einzelläufe (je 12 pro Abszissenwert); farbige Punkte mit Linienzug: Mittelwert; senkrechte Balken: Mittelwert ± eine Standardabweichung.

Punkte und erscheinen dunkler. Darüber liegt jeweils ein farbiger Punkt für den Mittelwert (links violett, rechts orange), der durch einen Linienzug mit den Nachbarpositionen verbunden ist, sowie ein senkrechter Balken mit Querstrichen, der von $\bar{t} - s$ bis $\bar{t} + s$ reicht. Die Legende steht nur im linken Teilbild.

Linkes Teilbild: Target-AND. Der Linienzug verläuft nicht monoton. Er beginnt mit dem kleinsten Mittelwert bei $m = 4$ (17,49 ms), springt bei $m = 6$ auf 24,23 ms, fällt über 21,25 ms ($m = 8$) auf ein Nebenminimum von 19,09 ms bei $m = 10$, bewegt sich danach zwischen 21,6 und 23,6 ms und erreicht bei $m = 20$ mit 24,29 ms seinen höchsten Wert. Über den gesamten Bereich, in dem die Instanz von 7 auf 39 Gates wächst, ändert sich der Mittelwert damit um höchstens 6,8 ms, also um weniger als eine Standardabweichung. Die Balken sind mit $s = 6{,}02$ bis 8,56 ms so lang, dass sich alle neun überlappen; der Balken bei $m = 4$ reicht beispielsweise von 11,35 bis 23,63 ms, der bei $m = 20$ von 16,64 bis 31,94 ms. Die grauen Punkte zeigen, woher die Streuung stammt: An fast jeder Eingangszahl liegt ein Teil der Läufe in einem schmalen Band bei 15,6 bis 16 ms, ein anderer Teil in einem Band bei etwa 31,7 bis 32 ms. Der Mittelwert fällt deshalb meist in die Lücke zwischen den beiden Bändern, in der nur wenige Einzelwerte liegen. Aus demselben Grund sind die Balken nicht als Konfidenzintervall zu lesen: Bei einer zweigipfligen Verteilung beschreibt $\bar{t} \pm s$ keinen Bereich, in dem die Läufe typischerweise liegen. Die Extremwerte sind 7,62 ms ($m = 4$) und 35,50 ms ($m = 10$). Ein mit m ansteigender Verlauf, wie ihn eine wachsende Instanz erwarten ließe, ist nicht erkennbar. Da bei Target-AND die Gate-Anzahl $2m - 1$ beträgt, ist die Rangkorrelation mit m identisch zu der mit der Gate-Anzahl ($\rho = 0{,}106$, $p = 0{,}275$).

Rechtes Teilbild: Target-AND-Random. Auch hier ist der Linienzug unsystematisch. Die Mittelwerte liegen für $d = 1$ bis 4 nahezu auf einem Plateau (22,70 bis 23,48 ms), erreichen bei $d = 5$ ihr Maximum von 26,57 ms, fallen bei $d = 6$ auf 20,77 ms, steigen bei $d = 7$ und $d = 9$ wieder auf rund 23 bis 24 ms und haben bei $d = 8$ ihr Minimum von 18,79 ms. Für $d = 10$ bis 12 liegen sie zwischen 19,85 und 21,99 ms, also im Mittel unter dem Plateau der kleinen Tiefen; bei $d = 12$ (20,56 ms) ist der Mittelwert um 2,52 ms

kleiner als bei $d = 1$ (23,08 ms), obwohl der Circuit dann doppelt so viele Gates hat. Die Extrema lassen sich an den Einzelpunkten nachvollziehen: Bei $d = 5$ liegt mehr als die Hälfte der Läufe oberhalb von 27,4 ms (Median 27,43 ms), bei $d = 8$ erreicht dagegen kein Lauf das obere Band (Maximum 28,66 ms), was den niedrigsten Mittelwert erklärt. Die tiefsten Einzelwerte der gesamten Studie stehen bei $d = 10$ (6,02 ms) und $d = 12$ (7,91 ms), der höchste (40,27 ms) bei $d = 2$. Die Balken ($s = 4{,}74$ bis 9,30 ms) überlappen ebenfalls durchgehend. Die lineare Regression ergibt $-0{,}28$ ms pro Schicht; über alle elf Tiefenschritte wären das $-3{,}0$ ms, etwa 0,4 Standardabweichungen aller Läufe. Da die Zwischenschichten mit festem Seed erzeugt werden, ist jeder Circuit deterministisch. Die Streuung innerhalb einer Tiefe ist daher reines Messrauschen und keine Variation zwischen verschiedenen Zufallscircuits.

Gesamtbild. In beiden Teilbildern bewegen sich die Mittelwerte im selben Band von etwa 17 bis 27 ms, und in beiden bilden die grauen Punkte dieselben zwei horizontalen Bänder bei 15,6 und 31,7 ms. Die Teilbilder unterscheiden sich damit weder in der Höhe noch in der Form der Streuung, obwohl links die Eingangszahl und rechts die Schichtenzahl variiert wird. Aus Abbildung 2 lässt sich daher kein Einfluss von m oder d auf die gemessene Laufzeit ablesen.

4.4 ISCAS'85-Benchmarks (F3)

Alle zwölf ISCAS'85-Circuits wurden in der Standard-, der Full- und der Performance-Suite je einmal ausgeführt (je Circuit $n = 3$, Tabelle 6). Die Instanzen reichen von 4 bis 3 512 Gates. Die mittleren Laufzeiten liegen zwischen 15,79 ms (c3540) und 31,85 ms (c1355). Der größte Circuit c7552 (3 512 Gates, 10 690 Klauseln) wird mit im Mittel 21,26 ms gelöst und ist damit nicht langsamer als der Mittelwert über alle ISCAS-Läufe (20,23 ms). Die Korrelation zwischen Gate-Anzahl und Laufzeit ist je Suite schwach und nicht signifikant (Standard: $r = 0{,}37$, $p = 0{,}24$; Full: $r = -0{,}18$, $p = 0{,}58$; Performance: $r = -0{,}20$, $p = 0{,}53$); über alle 36 ISCAS-Läufe gilt $\rho = 0{,}104$ ($p = 0{,}547$).

Circuit	Eingänge	Gates	Klauseln	n	$\bar{t}$	s	Min	Max
c3	4	4	14	3	15,80	0,07	15,75	15,88
c17	5	6	24	3	18,15	3,74	15,77	22,46
c432	36	160	616	3	16,50	1,27	15,70	17,96
c499	41	202	772	3	24,75	9,77	15,90	35,24
c880	60	383	1 354	3	18,64	2,60	15,72	20,72
c1355	41	546	2 020	3	31,85	0,16	31,69	32,00
c1908	33	880	2 495	3	19,10	5,50	15,91	25,45
c2670	233	1 193	3 553	3	19,18	11,22	11,54	32,06
c3540	50	1 669	5 118	3	15,79	0,18	15,67	16,00
c5315	178	2 307	7 480	3	20,51	4,00	16,01	23,66
c6288	32	2 416	9 600	3	21,22	8,90	16,06	31,49
c7552	207	3 512	10 690	3	21,26	9,45	15,70	32,18

Tabelle 6: ISCAS'85: Struktur und Laufzeit (ms) je Circuit, aufsteigend nach Größe. Je Circuit $n = 3$ Läufe (Standard, Full, Performance).

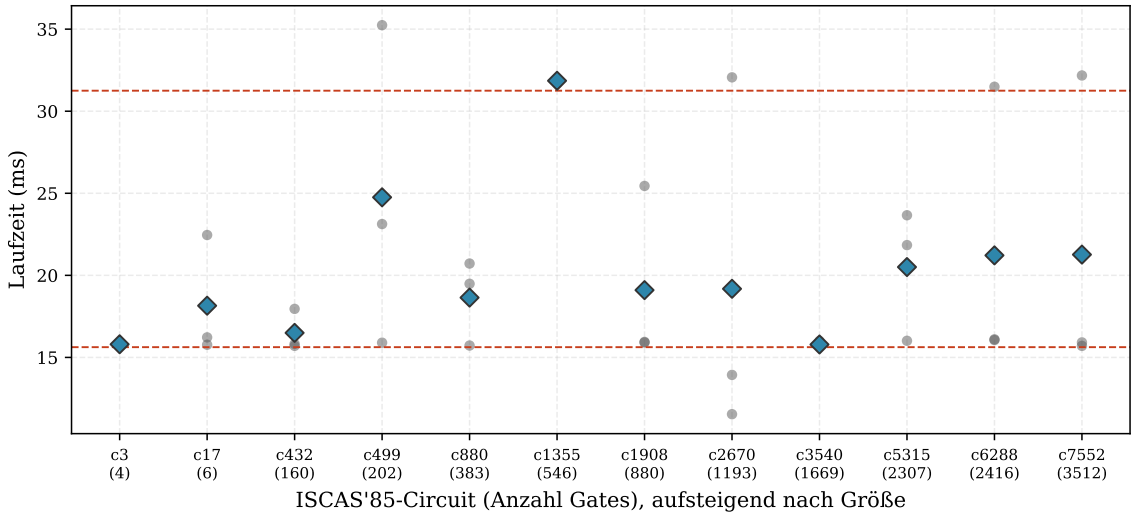


Abbildung 3: ISCAS'85: Einzelläufe (graue Punkte, je Circuit drei) und Mittelwert (blaue Raute) je Circuit. Die Circuits stehen in gleichen Abständen, aufsteigend nach Gate-Anzahl (in Klammern). Gestrichelt: 15,625 ms und 31,25 ms.

Aufbau der Abbildung. Abbildung 3 zeigt für jeden der zwölf ISCAS'85-Circuits die drei Einzelläufe (Standard, Full, Performance). Die Abszisse ist kategorial: Die Circuits stehen in gleichen Abständen, aufsteigend nach Gate-Anzahl von c3 (4 Gates) bis c7552 (3 512 Gates); der Abstand zweier Positionen ist daher *nicht* proportional zum Größenunterschied. Unter jedem Namen steht in Klammern die Gate-Anzahl. Die Ordinate gibt die Laufzeit in ms an. Die drei Läufe eines Circuits sind graue, halbtransparente Punkte direkt übereinander; fallen Werte zusammen, verschmelzen sie zu einem einzigen, dunkleren Punkt. Die blaue Raute markiert den Mittelwert, die beiden gestrichelten horizontalen Linien liegen bei 15,625 und 31,25 ms.

Lesart. Bei c3 und c3540 liegen alle drei Punkte praktisch übereinander auf der unteren Linie (15,75 bis 15,88 ms bzw. 15,67 bis 16,00 ms); die Raute sitzt dort bei 15,80 bzw. 15,79 ms. Das ist bemerkenswert, weil zwischen diesen beiden Circuits ein Faktor 417 in der Gate-Anzahl liegt (4 gegenüber 1 669). Bei zehn der zwölf Circuits liegt der kleinste der drei Werte zwischen 15,67 und 16,06 ms, also auf oder knapp über der unteren Linie, unabhängig davon, ob der Circuit 4 oder 3 512 Gates hat. Die beiden Ausnahmen sind c1355 (kleinster Wert 31,69 ms) und c2670 (11,54 ms, unterhalb der unteren Linie). Eine zweite Gruppe von Punkten liegt auf oder nahe der oberen Linie: je ein Lauf bei c2670 (32,06 ms), c6288 (31,49 ms) und c7552 (32,18 ms) sowie alle drei Läufe von c1355. Insgesamt liegen 19 der 36 Werte innerhalb von ±1 ms um die untere und 6 innerhalb von ±1 ms um die obere Linie.

Die größte Spannweite innerhalb eines Circuits haben c2670 (11,54 bis 32,06 ms, 20,5 ms) und c499 (15,90 bis 35,24 ms, 19,3 ms). Der Wert 35,24 ms bei c499 ist der einzige ISCAS-Lauf deutlich oberhalb der oberen Linie. Dadurch liegt die Raute von c499 (24,75 ms) höher als die der größeren Circuits c880 (18,64 ms) und c1908 (19,10 ms). Die Rauten der drei größten Circuits c5315, c6288 und c7552 liegen bei 20,51, 21,22 und 21,26 ms und damit auf demselben Niveau wie die der mittleren Circuits c880 bis c2670 (18,64 bis 19,18 ms). Von links nach rechts ist in der Folge der Rauten keine steigende Tendenz erkennbar.

Zusammenfassung des Bildes. Weder die Lage der Einzelpunkte noch die der Rauten ordnet sich nach der Gate-Anzahl. Die Punkte der größten Circuits rechts im Bild liegen im selben Wertebereich wie die der kleinsten links; ihre Streuung ist dabei teils größer als die der kleinen Circuits, jedoch ohne klare Ordnung. Das Bild zeigt damit dieselben zwei Bänder bei 15,625 und 31,25 ms wie Abbildung 1.

Auffällig ist c1355: In allen drei Läufen liegt die Laufzeit bei 31,69 bis 32,00 ms (Standardabweichung 0,16 ms), während die kleinere Instanz c880 (383 Gates) und die größere Instanz c1908 (880 Gates) im Mittel nur 18,64 ms bzw. 19,10 ms benötigen. Es ist der einzige Circuit mit einer in allen Wiederholungen übereinstimmend erhöhten Laufzeit. Mit nur drei Wiederholungen und der in Abschnitt 4.7 beschriebenen Messauflösung lässt sich nicht entscheiden, ob dies auf eine strukturelle Eigenschaft der Instanz oder auf Zufall zurückgeht. Auch die Funktion von c1355 als NAND-Variante von c499 (mit 24,75 ms im Mittel) liefert dafür keine eindeutige Erklärung.

4.5 Zusammenhang zwischen Instanzgröße und Laufzeit

Tabelle 7 fasst die Zusammenhangsmaße zusammen. Über alle 288 Läufe beträgt die Rangkorrelation zwischen Gate-Anzahl und Laufzeit $\rho = 0{,}024$ ($p = 0{,}686$); die lineare Regression ergibt eine Steigung von $-0{,}6\,\mu$s pro Gate bei $R^2 = 0{,}002$. In keiner Teilmenge ist ein Zusammenhang signifikant ($p > 0{,}1$ in allen Fällen), und die Bestimmtheitsmaße liegen unter 2,5 %. Abbildung 4 zeigt dies grafisch: Über mehr als zwei Größenordnungen der Gate-Anzahl verteilen sich die Laufzeiten im selben Band zwischen etwa 6 und 40 ms.

Teilmenge	n	ρ	p_ρ	r	p_r	Steigung (μs/Gate)	R^2
Alle Läufe	288	0,024	0,686	−0,047	0,427	−0,6	0,002
Target-AND	108	0,106	0,275	0,154	0,111	112,4	0,024
Target-AND-Random	144	−0,086	0,306	−0,129	0,124	−110,1	0,017
ISCAS'85	36	0,104	0,547	0,035	0,840	0,2	0,001
Target-AND-Random (Tiefe d)	144	−0,086	0,306	−0,134	0,109	–	0,018

Tabelle 7: Zusammenhang zwischen Instanzgröße (Gates bzw. Tiefe) und Laufzeit. ρ: Spearman, r: Pearson, jeweils mit p-Wert; Steigung und R^2 aus der linearen Regression der Laufzeit auf die Gate-Anzahl.

Aufbau der Abbildung. Abbildung 4 ist ein Streudiagramm, in dem jeder der 288 Läufe ein Punkt ist. Die Abszisse zeigt die Gate-Anzahl auf logarithmischer Skala (von 4 bis 3 512, knapp drei Größenordnungen), die Ordinate die Laufzeit in ms (linear, etwa von 5 bis 42 ms). Die Farbe kodiert den Circuit-Typ: Target-AND violett, Target-AND-Random orange, ISCAS'85 blau. Die Punkte sind halbtransparent. Weil viele Läufe dieselbe Gate-Anzahl haben (jeder Circuit wird bis zu fünfmal ausgeführt), wurde jeder Punkt horizontal um etwa 1 % der Abszisse zufällig verschoben; ein Punkt steht daher nur ungefähr über seiner Gate-Anzahl. Eine Ausgleichsgerade ist nicht eingezeichnet.

Die drei Punktwolken. Die Typen belegen verschiedene Abschnitte der Abszisse. Die Target-AND-Läufe (violett) liegen zwischen 7 und 39 Gates, die Target-AND-Random-Läufe (orange) zwischen 27 und 54 Gates und bilden eine dichte, schmale Säule; beide überlappen im Bereich von 27 bis 39 Gates. Oberhalb von 100 Gates liegen ausschließlich ISCAS-Läufe (blau): zehn Circuits mit zusammen 30 Läufen bis 3 512 Gates. Die übrigen

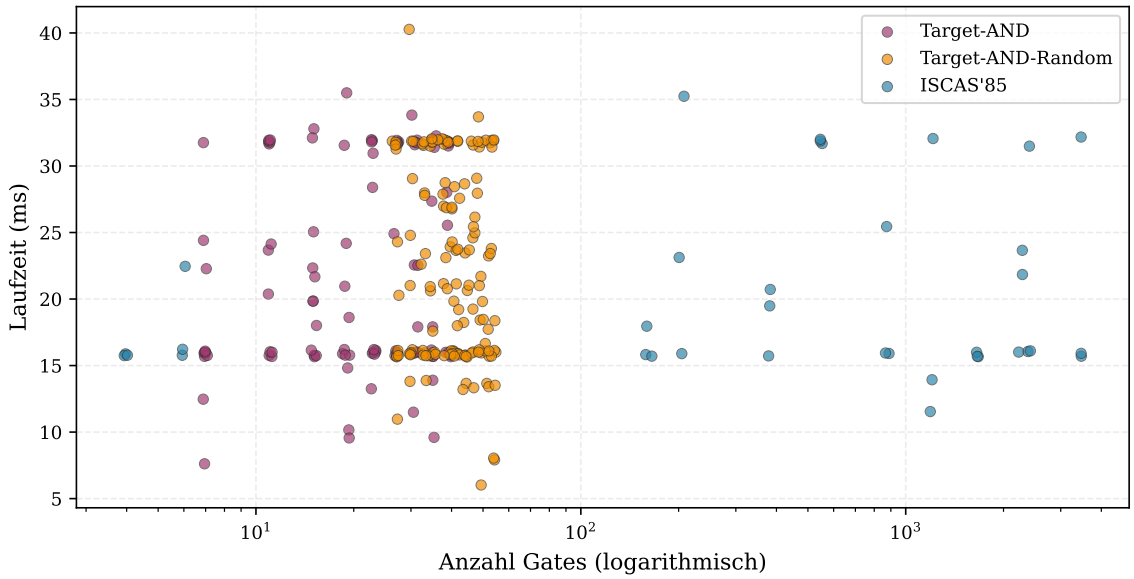


Abbildung 4: Laufzeit gegen Gate-Anzahl für alle 288 Läufe (logarithmische Abszisse, leichte horizontale Streuung der Punkte zur besseren Lesbarkeit). Violett: Target-AND, orange: Target-AND-Random, blau: ISCAS'85.

sechs ISCAS-Läufe (c3 und c17) liegen bei 4 bis 6 Gates links von allen synthetischen Läufen. Die Ordinatenwerte aller drei Wolken liegen im selben Bereich: bei Target-AND von 7,62 bis 35,50 ms, bei ISCAS85 von 11,54 bis 35,24 ms und bei Target-AND-Random von 6,02 bis 40,27 ms. Der höchste Punkt des Diagramms ist der Einzelwert 40,27 ms bei 30 Gates (orange); die tiefsten Punkte sind 6,02 und 7,91 ms bei 49 bzw. 54 Gates (ebenfalls orange).

Horizontale Bänder statt Anstieg. Auffällig sind zwei horizontale Bänder, auf denen sich die Punkte häufen: eines bei etwa 15,6 bis 16 ms und eines bei etwa 31,5 bis 32 ms. Beide ziehen sich über die gesamte Abszisse. Im unteren Band liegen die ISCAS-Punkte von c3 (4 Gates) bis c7552 (3 512 Gates), im oberen finden sich sieben ISCAS-Läufe bei 30 ms und mehr (unter anderem c1355 mit 546 Gates und c7552 mit 3 512 Gates). Zwischen den Bändern liegt ein dünner besiedelter Bereich, der überwiegend von den orangen Punkten gefüllt wird. Würde die Laufzeit mit der Instanzgröße wachsen, wäre eine von links unten nach rechts oben ansteigende Punktwolke zu erwarten; das Bild zeigt stattdessen, dass die Punkte rechts (bis 3 512 Gates) nicht höher liegen als links. Das deckt sich mit den Maßzahlen ($\rho = 0{,}024$ über alle Läufe, Tabelle 7). Weil zwischen den größten und kleinsten Instanzen derselbe Wertebereich liegt, bildet die Ordinate hier offenbar nicht die Lösungszeit ab, sondern die in Abschnitt 4.7 beschriebene Messgrundlast.

4.6 Vergleich der Circuit-Typen und der Suites (F4)

Die Circuit-Typen unterscheiden sich in ihrer Laufzeit nicht signifikant (Kruskal–Wallis: $H = 2{,}63$, $p = 0{,}268$); ebenso wenig die Gruppe der ISCAS-Läufe von den synthetischen Läufen (Mann–Whitney: $p = 0{,}140$). Auch zwischen den vier Suites besteht kein signifikanter Unterschied ($H = 2{,}39$, $p = 0{,}495$, Tabelle 9). Die Mittelwerte über die gesamte Suite betragen 24,52 ms (Quick), 22,46 ms (Standard), 21,68 ms (Full) und 21,27 ms (Performance); die Tendenz zu kleineren Mittelwerten bei größeren Suites ist mit den kleinen

Stichproben der Quick-Suite ($n = 21$) vereinbar.

Der direkte Vergleich der beiden gleich aufgebauten Suites Full und Performance (Tabelle 8) zeigt Abweichungen der Mittelwerte von +2,8 %, −5,6 % und +0,7 % für die drei Circuit-Typen; keiner dieser Unterschiede ist signifikant ($p = 0{,}513$, 0,483 und 0,751).

Circuit-Typ	$\bar{t}_{\text{Full}}$	$\bar{t}_{\text{Perf.}}$	Δ (ms)	Δ (%)	s_{Full}	$s_{\text{Perf.}}$
Target-AND	21,10	21,69	+0,60	+2,8 %	7,35	8,16
Target-AND-Random	22,61	21,34	−1,27	−5,6 %	7,02	7,04
ISCAS'85	19,23	19,38	+0,14	+0,7 %	5,71	6,27

Tabelle 8: Vergleich der Suites Full und Performance (identische Zusammensetzung). Mittelwerte $\bar{t}$ und Standardabweichungen s in ms.

Test	Prüfgröße	p-Wert
Kruskal–Wallis: Circuit-Typ (3 Gruppen)	$H = 2{,}63$	0,268
Kruskal–Wallis: Suite (4 Gruppen)	$H = 2{,}39$	0,495
Mann–Whitney: ISCAS'85 vs. synthetisch	$U = 3\,845$	0,140
Mann–Whitney: Full vs. Performance, Target-AND	$U = 931$	0,513
Mann–Whitney: Full vs. Performance, Target-AND-Random	$U = 1\,934$	0,483
Mann–Whitney: Full vs. Performance, ISCAS'85	$U = 66$	0,751

Tabelle 9: Ergebnisse der Rangtests. Alle p-Werte liegen deutlich über $\alpha = 0{,}05$.

Aufbau der Abbildung. Abbildung 5 fasst die Laufzeitverteilung je Suite und Circuit-Typ in Boxplots zusammen (die zugehörigen Zahlen stehen in Tabelle 3). Die Abszisse ist in drei Gruppen geteilt: Target-AND, Target-AND-Random und ISCAS'85. Innerhalb jeder Gruppe stehen von links nach rechts bis zu vier Boxen für die Suites Quick (blau), Standard (grün), Full (orange) und Performance (violett). Eine Box reicht vom ersten bis zum dritten Quartil und enthält damit die mittleren 50 % der Läufe; der schwarze Querstrich ist der Median. Die Whisker reichen bis zum äußersten Wert innerhalb des 1,5-fachen Interquartilsabstands, Werte außerhalb sind als kleine Kreise dargestellt. Die Ordinate zeigt die Laufzeit von 0 bis 48 ms. In der Gruppe ISCAS'85 fehlt die Quick-Box, weil Quick keine ISCAS-Circuits enthält (Tabelle 1); dort stehen nur drei Boxen, und die linke Position bleibt leer.

Target-AND. Die Boxen von Quick, Full und Performance sehen sich sehr ähnlich: Die Box reicht von rund 15,8 ms ($Q_1 = 15{,}77$ bis 15,90 ms) bis rund 31,5 ms ($Q_3 = 31{,}40$ bis 31,74 ms), und der Median liegt bei 16,00, 16,11 bzw. 16,20 ms unmittelbar am unteren Boxrand. Die Boxen sind also stark asymmetrisch: Mehr als die Hälfte der Läufe liegt im unteren Band, die obere Hälfte der Box ist dagegen sehr lang. Die Whisker reichen bei Full von 9,60 bis 32,80 ms, bei Performance von 7,62 bis 33,83 ms und bei Quick bis 35,50 ms. Abweichend ist nur Standard: Die Box ist schmal (22,56 bis 25,55 ms, Median 24,41 ms), und zwei Werte (15,89 und 30,96 ms) werden als Kreise abgesetzt. Dieser Unterschied beruht auf nur neun Läufen; zugleich ist es die Zelle mit der kleinsten Standardabweichung aller elf Kombinationen ($s = 4{,}30$ ms). Der Kruskal–Wallis-Test über alle Suites findet keinen signifikanten Unterschied ($p = 0{,}495$).

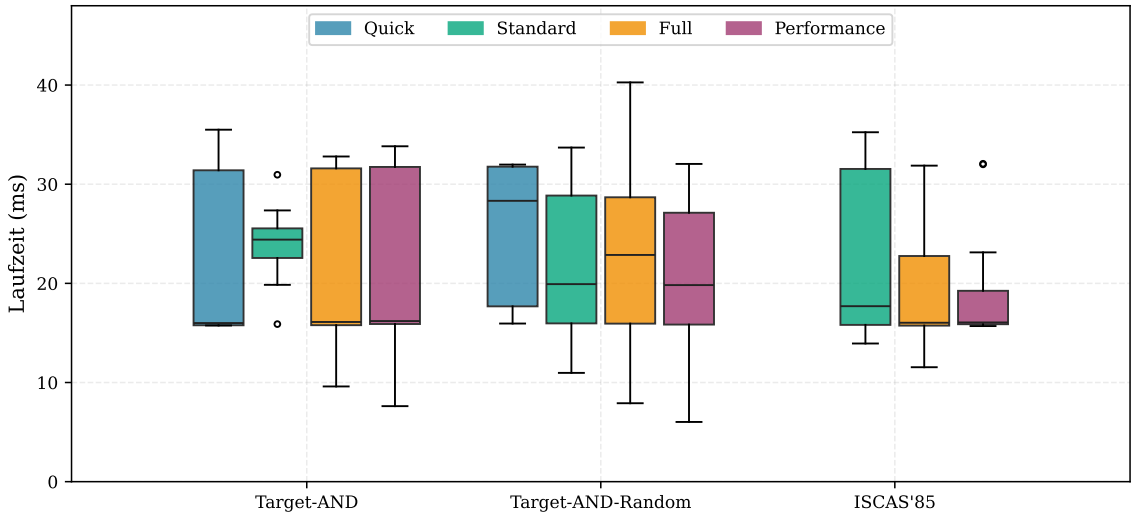


Abbildung 5: Boxplots der Laufzeit je Circuit-Typ (Gruppen) und Suite (innerhalb jeder Gruppe von links nach rechts: Quick blau, Standard grün, Full orange, Performance violett). Box: erstes bis drittes Quartil; Querstrich: Median; Whisker: äußerste Werte innerhalb des 1,5-fachen Interquartilsabstands; Kreise: Werte außerhalb. Für ISCAS'85 fehlt Quick (keine ISCAS-Läufe).

Target-AND-Random. Bei den zufälligen Circuits beginnen alle vier Boxen nahe dem unteren Band (Q_1 = 15,84 bis 15,96 ms; nur bei Quick 17,68 ms) und enden zwischen 27,12 ms (Performance) und 31,77 ms (Quick). Die Mediane streuen stärker als in den anderen Gruppen: Quick 28,32 ms, Standard 19,92 ms, Full 22,87 ms und Performance 19,83 ms. Der hohe Quick-Median ist der größte aller elf Mediane. Er liegt in der Lücke zwischen den beiden Bändern, wo die Dichte der Läufe gering ist; bei einer zweigipfligen Verteilung genügt dann die Verschiebung weniger Läufe zwischen den Bändern, um den Median stark zu verändern. Die Whisker von Full (7,91 bis 40,27 ms) und Performance (6,02 bis 32,05 ms) enthalten die Extremwerte der gesamten Studie.

ISCAS'85. Bei Full und Performance sind die Boxen verhältnismäßig schmal (15,74 bis 22,76 ms bzw. 15,88 bis 19,25 ms), und die Mediane liegen mit 16,03 und 16,06 ms am unteren Band; bei Performance ist ein Lauf mit 32,06 ms als Kreis abgesetzt. Die Standard-Box ist deutlich breiter (15,81 bis 31,54 ms, Median 17,70 ms, Whisker 13,93 bis 35,24 ms). In allen drei Suites liegen die ISCAS-Boxen nicht oberhalb der Boxen der kleinen synthetischen Circuits, obwohl die ISCAS-Instanzen bis zu etwa 90-mal mehr Gates haben als die Target-AND-Circuits.

Gesamtbild. Alle elf Boxen liegen zwischen 15,74 und 31,77 ms; ihre Mediane liegen zwischen 16,00 und 28,32 ms. Weder ordnen sich die vier Suitefarben innerhalb einer Gruppe systematisch (etwa aufsteigend), noch unterscheiden sich die drei Gruppen in Lage oder Form. Das entspricht den Rangtests in Tabelle 9 und dem Vergleich von Full und Performance in Tabelle 8. Die Unterschiede zwischen den Boxen sind kleiner als die Streuung innerhalb jeder einzelnen Box.

4.7 Zeitliche Auflösung der Messung

Tabelle 10 zeigt die Verteilung der Laufzeiten in Klassen. 105 der 288 Werte (36,5 %) liegen zwischen 15,0 und 16,5 ms und weitere 68 (23,6 %) zwischen 31,0 und 33,0 ms. Insgesamt liegen 173 Werte (60,1 %) innerhalb von ±1 ms um 15,625 ms oder 31,25 ms. Bei einer gleichmäßigen Verteilung über den beobachteten Bereich von 6 bis 40 ms wären für diese beiden Fenster nur etwa 12 % der Werte zu erwarten.

Der Wert 15,625 ms = 1/64 s entspricht der typischen Aktualisierungsrate der Systemuhr unter Windows. Die auffällige Häufung bei diesem Wert und seinem Doppelten ist daher konsistent mit einer groben Zeitauflösung von `time.time()`. Die Werte dazwischen (etwa 40 %) zeigen, dass die Auflösung nicht in allen Läufen gleich grob war. Wir behandeln diese Erklärung als gut gestützte Hypothese, nicht als bewiesen; der Datensatz enthält keine Information über Betriebssystem oder Uhr.

Laufzeitbereich (ms)	Anzahl	Anteil
(0,0; 10,0]	6	2,1 %
(10,0; 15,0]	17	5,9 %
(15,0; 16,5]	105	36,5 %
(16,5; 20,0]	20	6,9 %
(20,0; 30,0]	66	22,9 %
(30,0; 31,0]	1	0,3 %
(31,0; 33,0]	68	23,6 %
(33,0; 45,0]	5	1,7 %

Tabelle 10: Häufigkeit der Laufzeiten in Klassen über alle 288 Läufe.

4.8 Solver-Ausgabe und Erfüllbarkeit

In keinem der 288 Läufe wurde ein Erfüllbarkeitsergebnis erfasst: Das Feld `satisfiable` ist durchgehend `False`, das Feld `solver_output` stets leer, und es wurde kein Fehler protokolliert. Der Solver-Prozess wurde also gestartet und beendete sich ohne Ausgabe auf Standardausgabe und Fehlerausgabe, ohne dass der Runner den Rückgabecode ausgewertet hätte. Dieser Befund steht im Widerspruch zu der in Abschnitt 3.2 gezeigten Eigenschaft, dass die erzeugten Formeln erfüllbar sind. Er wird in Abschnitt 5 bewertet.

5 Diskussion

5.1 Was die Daten stützen

- **Keine Größenabhängigkeit im gemessenen Bereich.** Weder Gate-Anzahl, Klauselzahl, Eingangszahl noch Tiefe erklären die Laufzeit; alle Korrelationen sind schwach ($|\rho| \leq 0{,}11$) und nicht signifikant. Dies gilt über einen Größenbereich von 4 bis 3 512 Gates.
- **Kein Typ-Effekt.** ISCAS'85 mit durchschnittlich 1 106,5 Gates wird nicht langsamer verarbeitet als Target-AND mit durchschnittlich 23,0 Gates.
- **Konsistenz der Messungen.** Die Suites unterscheiden sich nicht signifikant, und Full und Performance liefern im Mittel dieselben Ergebnisse.

– **Stabilität.** Alle Läufe endeten ohne Timeout (30 s) und ohne Fehlermeldung; die maximale Laufzeit beträgt 40,27 ms.

5.2 Was die Daten nicht stützen

Eine Beobachtung der Form „die Laufzeit ist nicht von der Größe abhängig“ ist nur dann eine Aussage über den *Algorithmus*, wenn die gemessene Zeit tatsächlich die Rechenzeit des Algorithmus widerspiegelt. Dies ist hier aus vier Gründen nicht gesichert:

1. **Der Messwert wird vom Prozessstart dominiert.** Der häufigste Messwert liegt bei etwa 16 ms, und die Werte ändern sich nicht, wenn die Instanz um den Faktor 878 wächst: Der kleinste ISCAS-Circuit c3 (4 Gates) und der größte, c7552 (3 512 Gates), erreichen beide ein Minimum von etwa 15,7 bis 15,8 ms. Plausibel ist, dass die gemessene Zeit im Wesentlichen aus Prozessstart, Datei-I/O und Laufzeitumgebung besteht und die eigentliche Lösungszeit darunter liegt. Die Messung liefert dann eine obere Schranke der Solver-Zeit, aber keine Skalierungsinformation.

2. **Die Zeitauflösung ist grob** (Abschnitt 4.7). Der mehrgipfelige Verlauf der Verteilung, die in früheren Auswertungen als bimodale Verteilung langsamer und schneller Instanzen gedeutet wurde, ist mit einem Messartefakt gut vereinbar.

3. **Es wurde keine Solver-Ausgabe erfasst.** Weil für keinen Lauf ein SAT/UNSAT-Ergebnis vorliegt, ist nicht belegt, dass der Solver die Instanzen tatsächlich gelöst hat. Da die erzeugten Formeln erfüllbar sind (keine Unit-Klausel auf dem Ausgang, Abschnitt 3.2), müsste ein korrekt arbeitender Solver `SAT` melden; das durchgehend leere Ergebnis spricht dafür, dass entweder der Solver keine oder eine nicht erkannte Ausgabe erzeugt oder dass er die Eingabe nicht verarbeitet. Der Runner prüft den Rückgabecode des Prozesses nicht.

4. **Die Kodierung ist für NAND, NOR und XOR fehlerhaft** (Abschnitt 3.2). Die ISCAS'85-Läufe messen daher den Solver auf Formeln, die nicht die ISCAS-Circuits darstellen. Strukturell bleiben sie in Größe und Klauselzahl vergleichbar, ihr logischer Inhalt ist jedoch ein anderer.

Aus diesen Gründen können die Experimente weder die theoretische Laufzeit $\Theta(n^3)$ des Subgraph Algorithmus bestätigen noch widerlegen. Insbesondere ist die Aussage eines sublinearen oder konstanten Skalierungsverhaltens *nicht* aus den Daten ableitbar: Konstanz im Rauschen einer Prozessstart-dominierten Messung ist keine Konstanz des Algorithmus. Das gilt ebenso für Aussagen über einen tiefenabhängigen Anstieg, eine ISCAS-spezifische Beschleunigung oder nichtmonotones Laufzeitverhalten (etwa c1355): All dies liegt im Rahmen des Messrauschens bzw. lässt sich mit drei Wiederholungen je Circuit nicht absichern.

5.3 Gültigkeitsbedrohungen

– **Interne Validität.** `time.time()` ist keine monotone, hochauflösende Uhr; `time.perf_counter()` wäre geeigneter. Pro Instanz gibt es 1 bis 12 Wiederholungen ohne Aufwärmphase und ohne Berücksichtigung von Cache- und Betriebssystemeffekten.

– **Konstruktvalidität.** Gemessen wird die Wallclock-Zeit eines Prozesses, nicht die Rechenzeit des Algorithmus. Das Erfüllbarkeitsergebnis ist nicht erfasst (s. o.). Die Kodierung von NAND, NOR und XOR ist fehlerhaft. Der Speicherverbrauch wird nicht gemessen.

– **Externe Validität.** Die erzeugten Instanzen sind erfüllbar und strukturell einfach (Target-AND ist ein einzelner AND-Baum); die ISCAS-Formeln enthalten ebenfalls keine Ausgangsbedingung. Es gibt keine unerfüllbaren Instanzen und keinen Vergleich mit etablierten SAT-Solvern.

– **Umgebung.** Der Datensatz enthält keine Angaben zu Betriebssystem, Hardware, Python-Version oder Solver-Version.

5.4 Verbesserungen des Messaufbaus

Wir schlagen vor, die folgenden Änderungen am Framework vorzunehmen und die Experimente zu wiederholen:

1. Korrektur der Tseitin-Kodierung für NAND, NOR und XOR und Test der Kodierung gegen die Wahrheitstabelle jedes Gate-Typs.

2. Auswertung von Rückgabecode und Ausgabe des Solvers; Erfassen von SAT/UNSAT und gegebenenfalls der Belegung; Prüfen der Belegung gegen den Circuit.

3. Verwendung von `time.perf_counter()` und, falls vom Solver unterstützt, einer solverinternen Zeitmessung; zusätzlich Messung der Prozessstart-Grundlast mit einer trivialen Instanz.

4. Mehr Wiederholungen je Instanz (z. B. ≥ 30) mit Aufwärmläufen sowie Angabe von Median und Konfidenzintervall.

5. Instanzen mit Unit-Klausel auf dem Ausgang, unerfüllbare Instanzen und deutlich größere Circuits (10^4 bis 10^5 Gates), damit ein Skalierungseffekt über das Messrauschen hinauswächst.

6. Erfassen von Betriebssystem, Hardware, Python- und Solver-Version sowie Messung des Speicherverbrauchs.

7. Vergleich mit etablierten Solvern (z. B. MiniSat, CaDiCaL) auf denselben CNF-Dateien.

6 Neue Version der Experimente (Version 2.0.0)

Hinweis. Dieser Abschnitt berichtet über die *neue Version der Experimente*. Sie wurde mit der Version 2.0.0 der Python-Module des Frameworks (`subgraph-sat-solver-experiments`, `version = "2.0.0"` in `pyproject.toml`) am 11. Oktober 2026 ausgeführt und unterscheidet sich in Messaufbau, Instanzen und Auswertung von der ersten Messreihe (14. Juli 2026), die in den Abschnitten 3 bis 5 dokumentiert ist. Die Zahlen und Abbildungen der beiden Messreihen dürfen *nicht* ohne Weiteres miteinander verglichen werden (Abschnitt 6.1).

Die Diskussion der ersten Messreihe (Abschnitt 5) kam zu dem Schluss, dass deren Messwerte durch Prozessstart und grobe Zeitauflösung dominiert wurden, dass in keinem Lauf ein Solver-Ergebnis erfasst wurde und dass die Kodierung für NAND, NOR und XOR fehlerhaft war. Sie schlug sieben Verbesserungen des Messaufbaus vor (Abschnitt 5, Verbesserungen des Messaufbaus). Die Version 2.0.0 des Frameworks setzt diese Vorschläge um. Die in diesem Abschnitt beschriebenen Experimente sind die erste Messreihe mit diesem neuen Aufbau. Sie liegt als *Zwischenstand* des Full-Presets vor: Der Lauf wurde bei der Instanz `TargetAND_m10000` abgebrochen (Abschnitt 6.3.4).

6.1 Einordnung und Abgrenzung zur ersten Messreihe

Tabelle 11 stellt die Unterschiede zwischen erster Messreihe und neuer Version zusammen. Entscheidend für die Interpretation der folgenden Ergebnisse sind vier Punkte:

1. **Der Messwert ist nun eine Laufzeit mit Nanosekundenauflösung.** Die erste Messreihe verwendete `time.time()`; ihre Werte häuften sich bei 15,625 ms und 31,25 ms (Abbildung 1). Die neue Version verwendet `time.perf_counter_ns`. Die gemessenen Mediane (z. B. 9,52, 8,53, 10,11, 13,01 ms für $m = 4, 6, 8, 10$, Tabelle 14) liegen nicht auf den Zeittakten 15,625 ms oder 31,25 ms. Ob die Einzelläufe noch Häufungen zeigen, lässt sich im Zwischenstand nicht prüfen, weil nur Mediane vorliegen (Abschnitt 6.3.4).

2. **Es wird ein Ergebnis erfasst und geprüft.** Jeder Lauf erhält einen Status (`SAT`, `UNSAT`, `TIMEOUT`, `ERROR`, `NO_OUTPUT`); ein Modell wird gegen die CNF und gegen den Circuit geprüft. In der ersten Messreihe war für alle 288 Läufe weder Ausgabe noch Ergebnis erfasst worden (Abschnitt 4, Solver-Ausgabe und Erfüllbarkeit).

3. **Die Instanzen sind inhaltlich andere Formeln.** Die Tseitin-Kodierung wurde korrigiert (NAND, NOR, XOR), der Ausgang des Circuits wird per Unit-Klausel auf wahr gesetzt, und es gibt unerfüllbare Instanzen (Miter) sowie größere Circuits. Die Formeln der neuen Version sind daher nicht mit denen der ersten Messreihe identisch, auch wenn sie dieselben Namen tragen (z. B. `c17`, `TargetAND_m10`).

4. **Zwei Solver werden auf denselben CNF-Dateien gemessen.** Neben dem Subgraph-SAT-Solver läuft ein in Python geschriebener CDCL-Referenzsolver. Er dient der Korrektheitsprüfung und als grober Vergleich, ist aber ausdrücklich *nicht* auf Geschwindigkeit ausgelegt (Abschnitt 6.3.2).

Da die Umgebung der ersten Messreihe nicht dokumentiert wurde (Abschnitt 5, Gültigkeitsbedrohungen), lässt sich nicht beurteilen, wie viel eines Laufzeitunterschieds zwischen den Reihen auf die Hardware zurückgeht. Ein quantitativer Vergleich der absoluten Zeiten ist deshalb nicht möglich. Verglichen werden können nur qualitative Befunde (Abschnitt 6.5.4).

6.2 Hardware- und Softwareumgebung

Alle Messungen der neuen Version wurden am 11. Oktober 2026 auf einer virtuellen Maschine in der Cloud ausgeführt, nicht auf einem physischen Rechner. In derselben Umgebung liefen die Tests und Experimente der Projekte `subgraph-sat-solver`, `csubgraph`

Aspekt	Erste Messreihe (14. Juli 2026)	Neue Version 2.0.0 (11. Oktober 2026)
Zeitmessung	`time.time()`, Auflösung praktisch 15,625 ms	`time.perf_counter_ns`, Auflösung 1 ns
Ergebnis	nicht erfasst (`satisfiable` stets `False`)	Status je Lauf, Rückgabecode, Modellprüfung
Kodierung	NAND, NOR, XOR fehlerhaft; BUFF ohne Klauseln	alle Gate-Typen gegen Wahrheitstabellen getestet; BUFF, XNOR, n-äres XOR
Ausgangsbedingung	keine, Formeln stets erfüllbar	Unit-Klausel auf den ersten Ausgang
Unerfüllbare Instanzen	keine	Miter-Instanzen (Selbst-Miter, c499 gegen c1355)
Größte Instanz	3 512 Gates (c7552)	Skalierungsinstanzen bis $m = 50\,000$ vorgesehen
Wiederholungen	1 bis 12 je Instanz, ohne Warm-up	bis 30 je Instanz, 3 Warm-up-Läufe, Median
Timeout	30 s	60 s
Basislinie	keine	Prozessstart-Grundlast (triviale Instanz, $n = 30$)
Speicher	nicht gemessen	Spitzen-RSS je Lauf (`wait4`)
Umgebung	nicht dokumentiert	dokumentiert (Abschnitt 6.2)
Vergleichssolver	keiner	Python-CDCL-Referenzsolver

Tabelle 11: Unterschiede zwischen der ersten Messreihe und der neuen Version 2.0.0 der Experimente.

und `subgraph-sat-solver-experiments`. Tabelle 12 fasst die Konfiguration zusammen; die Quelle ist die Datei `src/results/hardware.md`.

Für die Interpretation der Messwerte sind vier Eigenschaften dieser Umgebung wichtig:

- **Ein einziger Kern.** Der Solver-Prozess und der Messprozess (Python-Interpreter des Frameworks mit dem Timer-Thread für das Zeitlimit) teilen sich die eine CPU. Das kann die gemessenen Laufzeiten leicht verzerren, vor allem bei sehr kurzen Läufen im Bereich weniger Millisekunden.
- **Wenig Arbeitsspeicher und kein Swap.** Die Adjazenzmatrix der Reduktion wächst quadratisch mit der Zahl der Literalvorkommen (Abschnitt 2.2). Bei sehr großen Instanzen kann der Speicher von 3,9 GiB knapp werden; ohne Swap kann eine Überschreitung zum Abbruch des Prozesses führen. Ob dies die Ursache der in Abschnitt 6.4 beschriebenen `ERROR`-Läufe und des Abbruchs des Full-Laufs war, wurde nicht geprüft.
- **Build ohne `-march=native`.** Die Binaries wurden direkt mit `g++` und `-O2` gebaut. Die CMake-Konfiguration des Solvers setzt dagegen `-O3 -march=native`. Die Laufzeiten sind daher mit Messungen aus einem CMake-Build nur eingeschränkt vergleichbar; ein Build mit `-O3 -march=native` könnte den Subgraph-SAT-Solver schneller machen.

Merkmal	Wert
Prozessor	
Architektur	x86_64, 64 Bit (32-Bit-Code ausführbar), Little Endian
Modell	Intel(R) Xeon(R) Processor @ 2,80 GHz
Sockets / Kerne / Threads	1 Socket, 1 Kern pro Socket, 1 Thread pro Kern; **1 CPU insgesamt**
Adressbreite	46 Bit physisch, 48 Bit virtuell
Cache	L2: 1 MiB (1 Instanz), L3: 33 MiB (1 Instanz)
Speicher	
Arbeitsspeicher	3,9 GiB
Swap	keiner
Software	
Betriebssystem	Linux, Kernel 6.18.44 (glibc 2.39)
Python	CPython 3.12.3
NumPy / Matplotlib	2.4.4 / 3.10.8
C++-Compiler	g++ (C++17, `-O2 -Wall -Wextra`)
Zeitgeber	`time.perf_counter_ns` (monoton, Auflösung 1 ns)

Tabelle 12: Hardware- und Softwareumgebung der neuen Experimente (virtuelle Maschine, 11. Oktober 2026).

– **Virtuelle Maschine.** Taktrate und Cache-Angaben sind die Werte, die die VM meldet. Die tatsächlich zugeteilte Rechenzeit kann schwanken. Die Messungen sind daher nicht als Messungen eines physischen Prozessors mit fester Taktrate zu lesen.

Die Version des Subgraph-SAT-Solvers selbst wurde im Zwischenstand nicht festgehalten, weil die Datei `environment.json` erst am Ende eines vollständigen Laufs geschrieben wird (Abschnitt 6.3.4).

6.3 Aufbau der neuen Experimente

6.3.1 Messablauf

Das Framework erzeugt für jede Instanz eine DIMACS-CNF-Datei und startet jeden Solver als eigenen Prozess (`subprocess.Popen`, Standard- und Fehlerausgabe in temporäre Dateien). Die Laufzeit t ist die Zeit zwischen `time.perf_counter_ns()` vor dem Start und dem Ende des Prozesses, in Millisekunden; sie enthält damit Prozessstart, Einlesen der Datei, Lösen und Prozessende. Beim Warten auf das Prozessende liest das Framework über `os.wait4` außerdem die Spitzen-Speichernutzung des Prozesses (`ru_maxrss`). Ein Timer-Thread beendet den Prozess nach Ablauf des Zeitlimits (Status `TIMEOUT`).

Der Status eines Laufs wird aus der Ausgabe und dem Rückgabecode bestimmt. Maßgeblich sind in dieser Reihenfolge eine `s`-Zeile im Format der SAT-Competition (`s SATISFIABLE`), die Zeile `Status:` des Subgraph-SAT-Solvers, eine alleinstehende Ergebniszeile und zuletzt der Rückgabecode (10: `SAT`, 20: `UNSAT`). Die `Status:`-Zeile hat Vorrang vor dem Rückgabecode, weil der Subgraph-SAT-Solver den Code 20 auch bei `UNKNOWN` und bei Fehlern liefert. Ein Rückgabecode außerhalb von $\{0, 10, 20\}$ ergibt immer `ERROR`; ein Lauf ohne erkennbares Ergebnis und mit Rückgabecode 0 ergibt `NO_OUTPUT`. Freie

Textsuche nach „SAT“ findet nicht mehr statt, weil sie in der ersten Messreihe auch auf „UNSAT“ und auf Banner-Zeilen zutraf.

Für jede Instanz und jeden Solver gilt folgender Ablauf. Zuerst werden 3 Warm-up-Läufe ausgeführt und verworfen. Die Zahl der gemessenen Wiederholungen ist

$$n \;=\; \max\Big(3,\; \min\big(30,\; \lfloor 60\,000 / t_w \rfloor\big)\Big),$$

wobei t_w die Laufzeit des letzten Warm-up-Laufs in Millisekunden ist. Der Zeitbudget-Term begrenzt den Aufwand je Instanz auf etwa 60 s: Schnelle Instanzen werden 30-mal gemessen, langsame nur mindestens dreimal. Endet schon ein Warm-up-Lauf mit `TIMEOUT`, `ERROR` oder `NO_OUTPUT`, wird der Warm-up abgebrochen und die Instanz nur noch einmal gemessen ($n = 1$). Berichtet wird der *Median* der Laufzeiten der n gemessenen Läufe. Läufe mit `TIMEOUT` gehen nicht in den Median der übrigen ein; bei $n = 1$ ist der angegebene Wert die Dauer dieses einen Laufs. Ein Timeout- oder Fehlerwert ist daher keine Median-Schätzung, sondern ein einzelner Messwert und bei `ERROR` die Zeit bis zum Fehler.

Zusätzlich wird je Solver zu Beginn eines Presets die *Basislinie* gemessen: 30 Läufe mit der trivialen Instanz `p cnf 2 2` mit den Klauseln $(x_1 \vee x_2)$ und $(\neg x_1 \vee x_2)$. Ihr Median ist die Prozessstart-Grundlast, die in allen Laufzeiten enthalten ist.

6.3.2 Die beiden Solver

Subgraph-SAT-Solver. Der Solver ist das in Abschnitt 2.2 beschriebene Kommandozeilenprogramm (Reduktion von CNF auf 3-SAT, auf Clique und auf Subgraph-Isomorphismus; externes Subgraph-Programm). Er wurde mit `g++ -O2` gebaut (Abschnitt 6.2). Seine Laufzeit enthält deshalb außer der Reduktion auch Datei-I/O und den Start des externen Subgraph-Prozesses.

Referenzsolver. Der Referenzsolver (`python -m src.reference_solver`) ist ein kleiner CDCL-Solver in reinem Python (Zwei-Watched-Literals-Propagation, 1-UIP-Lernen mit nicht-chronologischem Backjumping, VSIDS, Phase-Saving, Luby-Restarts). Er ist laut seiner Dokumentation nicht auf Geschwindigkeit ausgelegt und mit MiniSat oder CaDiCaL nicht konkurrenzfähig. Er liefert dasselbe Aufrufprotokoll wie ein externer Solver (Ausgabe im SAT-Competition-Format, Rückgabecode 10/20) und dient hier (i) als unabhängige Prüfung der SAT/UNSAT-Ergebnisse und (ii) als grober Vergleichsmaßstab. Weil er als Python-Programm startet, enthält jede seiner Laufzeiten die Startzeit des Python-Interpreters; sie ist in der Basislinie sichtbar (Abschnitt 6.4.1). *Messwerte dieses Solvers sagen nichts über die Leistung etablierter SAT-Solver aus.*

6.3.3 Instanzen

Das Full-Preset umfasst 46 Instanzen: 9 Target-AND-Circuits ($m = 4, 6, \ldots, 20$; Experiment 1), 12 Target-AND-Random-Circuits ($m = 10$, $d = 1, \ldots, 12$; Experiment 2), die 12 ISCAS’85-Circuits (Experiment 3), 9 Miter-Instanzen und vier Skalierungsinstanzen (Target-AND mit $m \in \{100, 1000, 10\,000, 50\,000\}$). Alle Instanzen verwenden die Ausgangsbedingung (`constraint=output`): Der erste Ausgang des Circuits wird durch eine Unit-Klausel auf wahr gesetzt. Dadurch sind die Formeln nicht mehr trivial erfüllbar.

Target-AND und Target-AND-Random. Konstruktion und Eingangszahlen sind die der ersten Messreihe (Abschnitt 3). Durch die geänderte Kodierung ändern sich aber Klausel- und Variablenzahl:

Satz 2 (Größe der Target-AND-Formeln der Version 2.0.0)**.** Die Formel eines Target-AND-Circuits mit $m \geq 2$ Eingängen und Ausgangsbedingung besitzt $3m - 1$ Variablen, $5m - 2$ Klauseln und $11m - 6$ Literalvorkommen.

Beweis. Der Circuit hat $2m - 1$ Gates (Satz 1); zusammen mit den m Eingängen ergeben sich $m + (2m - 1) = 3m - 1$ Variablen. Jedes der m NOT-Gates erzeugt 2 Klauseln mit je 2 Literalen, also $2m$ Klauseln und $4m$ Literale. Jedes der $m - 1$ AND-Gates mit zwei Eingängen erzeugt in der korrigierten Kodierung 3 Klauseln, nämlich $(\neg y \vee x_1)$, $(\neg y \vee x_2)$ und $(y \vee \neg x_1 \vee \neg x_2)$ mit zusammen $2 + 2 + 3 = 7$ Literalen; die in der ersten Messreihe zusätzlich erzeugte, redundante Klausel $(\neg y \vee x_1 \vee x_2)$ entfällt. Hinzu kommt die Unit-Klausel des Ausgangs. Insgesamt sind das $2m + 3(m - 1) + 1 = 5m - 2$ Klauseln und $4m + 7(m - 1) + 1 = 11m - 6$ Literale. □

Die Daten bestätigen dies: $m = 4$ ergibt 18 Klauseln und 38 Literale, $m = 20$ ergibt 98 Klauseln und 214 Literale, $m = 100$ ergibt 498 und 1 094, $m = 1000$ ergibt 4 998 und 10 994 (Tabelle 14). Der Satz 1 (mit $6m - 4$ Klauseln) gilt für die Kodierung der ersten Messreihe. Die Target-AND-Random-Circuits verwenden für alle fünf Gate-Typen die korrigierte Kodierung; ihre Größe hängt von den zufälligen Schichten ab (Tabelle 15).

ISCAS'85. Verwendet werden dieselben zwölf Circuits wie in der ersten Messreihe. Der Parser sortiert die Gates topologisch und meldet undefinierte Signale und Zyklen als Fehler. Die BUFF-Gates der `.bench`-Dateien (in der ersten Messreihe ohne Klauseln) werden nun kodiert. Als Ausgangsbedingung wird der erste Ausgang des Circuits auf wahr gesetzt; ob die Formel erfüllbar ist, ist vorab nicht bekannt.

Miter-Instanzen. Ein Miter verbindet zwei Circuits A und B mit gemeinsamen Eingängen; der Ausgang ist das ODER über die XOR-Verknüpfungen entsprechender Ausgänge. Mit der Ausgangsbedingung ist die Formel genau dann erfüllbar, wenn sich A und B auf mindestens einer Eingabe unterscheiden. Sind sie funktional äquivalent, ist die Formel *unerfüllbar*. Es gibt zwei Arten von Miter-Instanzen: *Selbst-Miter*, bei denen ein Circuit mit einer Kopie seiner selbst verglichen wird (Target-AND mit $m = 10$ und $m = 20$ sowie die ISCAS-Circuits c17, c432, c499, c880, c1355 und c1908), und ein *Äquivalenz-Miter* von c499 gegen c1355. Letzteres ist sinnvoll, weil c1355 die Variante von c499 mit NAND- statt XOR-Gates ist (Abschnitt 3). Alle neun Miter-Instanzen sind unerfüllbar. Ein Miter enthält die Gates beider Circuits und zusätzlich die XOR- und ODER-Gates; `MiterSelf_c1908` hat deshalb 1 786 Gates, c1908 selbst 880.

Skalierungsinstanzen. Target-AND mit sehr vielen Eingängen ($m = 100$ bis 50 000, bis zu 99 999 Gates) soll prüfen, ob ein Skalierungseffekt über das Messrauschen hinauswächst, wie in den Verbesserungsvorschlägen gefordert.

6.3.4 Datenbasis dieses Abschnitts

Der Lauf des Full-Presets wurde nicht vollständig beendet. Die Konsolenausgabe (`src/results/full_log_zwischenstand.log`) endet bei der Zeile für den Subgraph-SAT-Solver auf der Instanz `TargetAND_m10000`, ohne dass ein Ergebnis ausgegeben wurde.

Parameter des Full-Presets	Wert
Wiederholungen R (Obergrenze)	30
Warm-up-Läufe	3
Mindestzahl Wiederholungen	3
Zeitlimit je Lauf	60 s
Zeitbudget je Instanz B	60 s
Instanzen	46 (Exp. 1: 9, Exp. 2: 12, Exp. 3: 12, Miter: 9, Skalierung: 4)
Solver	Subgraph-SAT-Solver, Referenzsolver (CDCL, Python)

Tabelle 13: Konfiguration des Full-Presets der neuen Version 2.0.0.

Ob der Speicher von 3,9 GiB der Grund war, wurde nicht geprüft (Abschnitt 6.2). Daraus folgt:

- Ausgewertet werden 44 der 46 Instanzen, jeweils für beide Solver. Es fehlen `TargetAND_m10000` und `TargetAND_m50000`.
- Die Dateien, die erst am Ende eines Laufs geschrieben werden (`experiment_results.csv/.json`, `summary.csv`, `environment.json`, `baseline.json`, `config.json`, Diagramme), liegen nicht vor. Datenbasis ist die Datei `src/results/full_zwischenstand.csv` mit den Spalten `solver`, `instanz`, `gates`, `status`, `n`, `median_ms` und `anmerkung`. Ihre 88 Zeilen (44 Instanzen × 2 Solver) stimmen vollständig mit den 88 Ergebniszeilen der Konsolenausgabe überein; die Basislinien stehen nur in der Konsolenausgabe.
- Die CSV-Datei enthält je Instanz und Solver *nur den Median* und die Zahl n der gemessenen Läufe, aber nicht die Einzelläufe. Streumaße (Quartile, Konfidenzintervalle), Rückgabecodes, Ausgaben der Solver, Speicherverbrauch und das Ergebnis der Modellprüfung je Lauf sind deshalb nicht verfügbar. Aussagen über die Streuung beschränken sich auf die Basislinie (Standardabweichung s).
- Insgesamt gingen 933 gemessene Läufe des Subgraph-SAT-Solvers und 1 257 des Referenzsolvers in die Mediane ein (Summe der n, ohne Warm-up).

Tabellen und Abbildungen dieses Abschnitts erzeugt das Skript `src/paper_analysis_v2.py` direkt aus diesen Dateien (Abschnitt 6.6).

6.4 Ergebnisse der neuen Experimente

6.4.1 Basislinie: Prozessstart-Grundlast

Die Basislinie mit der trivialen Instanz ($n = 30$ Läufe je Solver) beträgt für den Subgraph-SAT-Solver 8,43 ms (Standardabweichung $s = 1{,}80$ ms) und für den Referenzsolver 56,18 ms ($s = 14{,}63$ ms). Der Referenzsolver braucht allein für Start und Beenden also etwa das 6,7-Fache. Der Unterschied rührt daher, dass der Python-Interpreter gestartet werden muss. Beide Werte sind in allen folgenden Laufzeiten enthalten und in den Abbildungen als gepunktete Linien eingezeichnet. Differenzen unterhalb weniger Millisekunden (Subgraph-SAT-Solver) bzw. unterhalb von etwa 15 ms (Referenzsolver,

Standardabweichung der Basislinie) sind deshalb nicht von der Grundlast zu unterscheiden. Dies erklärt, warum zwei Mediane des Subgraph-SAT-Solvers (c3: 6,98 ms, c17: 7,24 ms) und 30 von 44 Mediane des Referenzsolvers unterhalb der jeweiligen Basislinie liegen: Die Basislinie ist selbst ein Median mit Streuung und wurde einmal zu Beginn gemessen.

6.4.2 Gesamtüberblick

Abbildung 6 zeigt die Mediane aller Instanzen beider Solver in Abhängigkeit von der Gate-Anzahl, Abbildung 7 die Verteilung der Status je Instanzfamilie.

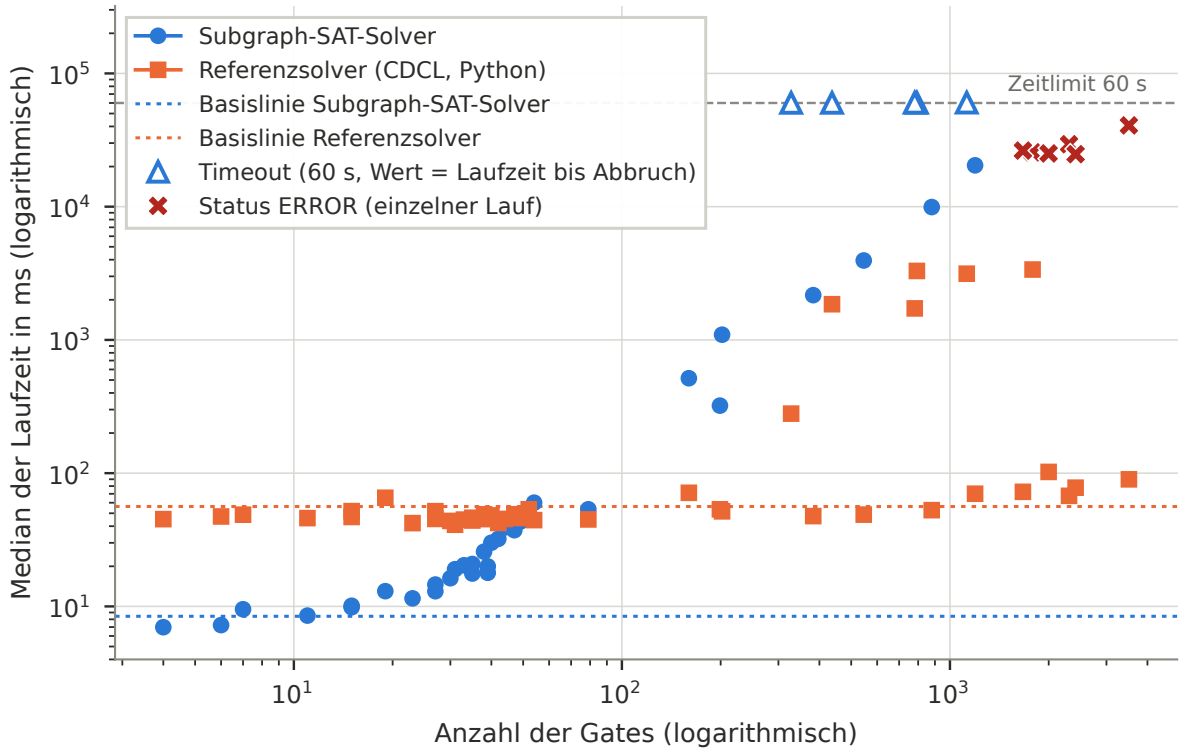


Abbildung 6: Median der Laufzeit aller 44 ausgewerteten Instanzen über der Gate-Anzahl (beide Achsen logarithmisch) für den Subgraph-SAT-Solver und den Referenzsolver der neuen Version 2.0.0.

Aufbau der Abbildung. Abbildung 6 ist ein Streudiagramm mit logarithmischer Abszisse (Gate-Anzahl von 4 bis 3 512) und logarithmischer Ordinate (Median in ms, von etwa 4 ms bis $3 \cdot 10^5$ ms). Jede Instanz erscheint zweimal, einmal je Solver. Blaue Kreise sind Mediane des Subgraph-SAT-Solvers, orange Quadrate Mediane des Referenzsolvers. Weiße, blau umrandete Dreiecke markieren Instanzen, bei denen der Subgraph-SAT-Solver das Zeitlimit von 60 s erreichte; ihr Wert ist die Dauer des einen Laufs bis zum Abbruch (60,04 bis 60,27 s). Rote Kreuze markieren Instanzen mit dem Status `ERROR` (einzelner Lauf, Wert ist die Zeit bis zum Fehler). Die beiden gepunkteten horizontalen Linien sind die Basislinien (8,43 ms blau, 56,18 ms orange), die gestrichelte graue Linie ist das Zeitlimit von 60 s. Da die Abszisse nur die Gate-Anzahl und nicht die Instanzfamilie kodiert, überlagern sich Punkte verschiedener Familien mit ähnlicher Gate-Anzahl.

Lesart: Subgraph-SAT-Solver. Die blauen Punkte bilden drei Gruppen. Bei höchstens 79 Gates (alle Target-AND-, Target-AND-Random-, c3-, c17- und die drei kleinen Miter-Instanzen) liegen sie zwischen 6,98 und 60,04 ms, nahe der Basislinie und mit leichtem Anstieg zu größeren Instanzen. Ab etwa 160 Gates steigen sie steil an: 515,75 ms bei c432 (160 Gates), 3 945,39 ms bei c1355 (546 Gates), 20 427,66 ms bei c2670 (1 193 Gates), der größten gelösten Instanz. Bei einem Faktor 7,5 in der Gate-Anzahl (160 bis 1 193) steigt die Laufzeit damit um den Faktor 39,6. Die dritte Gruppe sind die fünf Dreiecke (328 bis 1 125 Gates, ausschließlich Miter-Instanzen) und die sechs Kreuze (1 669 bis 3 512 Gates).

Lesart: Referenzsolver. Die orangen Quadrate liegen für alle Nicht-Miter-Instanzen in einem flachen Band zwischen 41,02 und 102,17 ms, in der Nähe der Basislinie von 56,18 ms, auch bei 3 512 Gates (c7552: 89,88 ms). Nur die sechs größeren Miter-Instanzen heben sich ab: Ihre Quadrate liegen zwischen 279,73 und 3 374,42 ms. Der Referenzsolver löst *alle* 44 Instanzen, und seine Laufzeit liegt, mit Ausnahme dieser sechs Instanzen, im gesamten Bereich nahe an der Interpreter-Startzeit.

Kreuzung der Kurven. Bei kleinen Instanzen (bis etwa 40 Gates) liegt der Subgraph-SAT-Solver deutlich unter dem Referenzsolver, weil sein Prozessstart nur etwa 8 ms statt 56 ms kostet. Bei etwa 50 Gates treffen sich die Kurven (Abschnitt 6.4.4); ab etwa 160 Gates liegt der Subgraph-SAT-Solver deutlich darüber. Dieser Schnittpunkt ist vor allem ein Effekt der unterschiedlichen Startkosten der beiden Programme und keine Aussage über die Qualität der Algorithmen (Abschnitt 6.4.7).

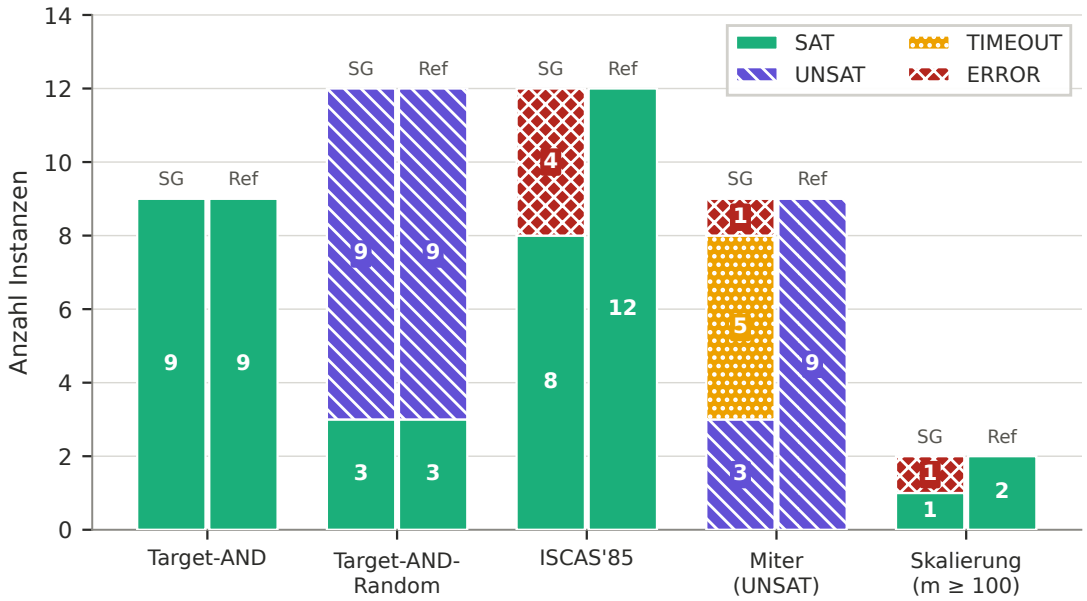


Abbildung 7: Anzahl der Instanzen je Instanzfamilie und Status (SAT, UNSAT, Timeout, ERROR) für den Subgraph-SAT-Solver (SG) und den Referenzsolver (Ref) der neuen Version 2.0.0.

Aufbau der Abbildung. Abbildung 7 ist ein gestapeltes Balkendiagramm. Für jede der fünf Instanzfamilien (Target-AND, Target-AND-Random, ISCAS'85, Miter, Skalierung

mit $m \geq 100$) stehen zwei Balken nebeneinander, links der Subgraph-SAT-Solver (SG), rechts der Referenzsolver (Ref). Die Höhe eines Balkens ist die Zahl der Instanzen der Familie, die Segmente zeigen die Anzahl je Status (grün: `SAT`, violett schraffiert: `UNSAT`, gelb gepunktet: Timeout, rot gekreuzt: `ERROR`); die Zahl im Segment ist die Instanzzahl.

Lesart. Der Referenzsolver erreicht bei allen 44 Instanzen ein Ergebnis: Target-AND 9× SAT, Target-AND-Random 3× SAT und 9× UNSAT, ISCAS'85 12× SAT, Miter 9× UNSAT, Skalierung 2× SAT. Der Subgraph-SAT-Solver erreicht bei 33 Instanzen ein Ergebnis, bei 5 ein Timeout und bei 6 den Status `ERROR`. Auf Target-AND und Target-AND-Random liefern beide Solver auf allen Instanzen dasselbe Ergebnis. Bei ISCAS'85 fallen 4 von 12 Instanzen (c3540, c5315, c6288, c7552) mit `ERROR` aus, bei den Miter-Instanzen sind es 5 Timeouts und 1 `ERROR`, bei den Skalierungsinstanzen ein `ERROR` (`TargetAND_m1000`). Der Subgraph-SAT-Solver löst damit alle erfüllbaren Instanzen bis 1 193 Gates und die kleinen unerfüllbaren Instanzen; an allen sechs Instanzen ab 1 669 Gates scheitert er mit `ERROR`.

6.4.3 Experiment 1: Target-AND und Skalierung

Tabelle 14 zeigt Struktur, Status und Mediane für die neun Target-AND-Circuits mit $m = 4, \ldots, 20$ und die beiden ausgewerteten Skalierungsinstanzen ($m = 100$ und $m = 1000$). Abbildung 8 stellt sie grafisch dar.

Instanz	Gates	Klauseln	Literale	Status	n_{SG}	$\tilde{t}_{\mathrm{SG}}$	n_{Ref}	$\tilde{t}_{\mathrm{Ref}}$
`TargetAND_m4`	7	18	38	SAT	30	9,52	30	48,79
`TargetAND_m6`	11	28	60	SAT	30	8,53	30	45,98
`TargetAND_m8`	15	38	82	SAT	30	10,11	30	51,91
`TargetAND_m10`	19	48	104	SAT	30	13,01	30	65,36
`TargetAND_m12`	23	58	126	SAT	30	11,49	30	42,18
`TargetAND_m14`	27	68	148	SAT	30	13,00	30	51,86
`TargetAND_m16`	31	78	170	SAT	30	19,02	30	41,02
`TargetAND_m18`	35	88	192	SAT	30	17,65	30	46,28
`TargetAND_m20`	39	98	214	SAT	30	17,88	30	45,20
`TargetAND_m100`	199	498	1 094	SAT	30	321,23	30	53,56
`TargetAND_m1000`	1 999	4 998	10 994	ERROR / SAT	1	25 087,83	30	102,17

Tabelle 14: Target-AND und Skalierung: Struktur, Status (Subgraph-SAT-Solver / Referenzsolver, falls verschieden) und Median $\tilde{t}$ der Laufzeit in ms (SG: Subgraph-SAT-Solver, Ref: Referenzsolver; n: Zahl der Messläufe).

Aufbau der Abbildung. Abbildung 8 hat zwei Teilbilder. In Teilbild (a) sind die neun Mediane für $m = 4, 6, \ldots, 20$ auf linearen Achsen aufgetragen (Ordinate 0 bis 72 ms), jeweils als Linienzug mit Punkten: blaue Kreise für den Subgraph-SAT-Solver, orange Quadrate für den Referenzsolver. Die gepunkteten Linien sind die Basislinien. Teilbild (b) enthält dieselben Punkte und zusätzlich $m = 100$ und $m = 1000$ auf doppeltlogarithmischen Achsen (Abszisse m, Ordinate 4 bis $1{,}2 \cdot 10^5$ ms). Bei $m = 1000$ endet der blaue Linienzug; dort steht ein rotes Kreuz mit der Beschriftung „ERROR (25,1 s)“, das den fehlgeschlagenen Lauf des Subgraph-SAT-Solvers kennzeichnet. Der orange Linienzug läuft bis $m = 1000$ durch.

Lesart von Teilbild (a). Der Median des Subgraph-SAT-Solvers steigt von 9,52 ms bei $m = 4$ nicht streng monoton, aber erkennbar auf 17,88 ms bei $m = 20$. Das Minimum liegt

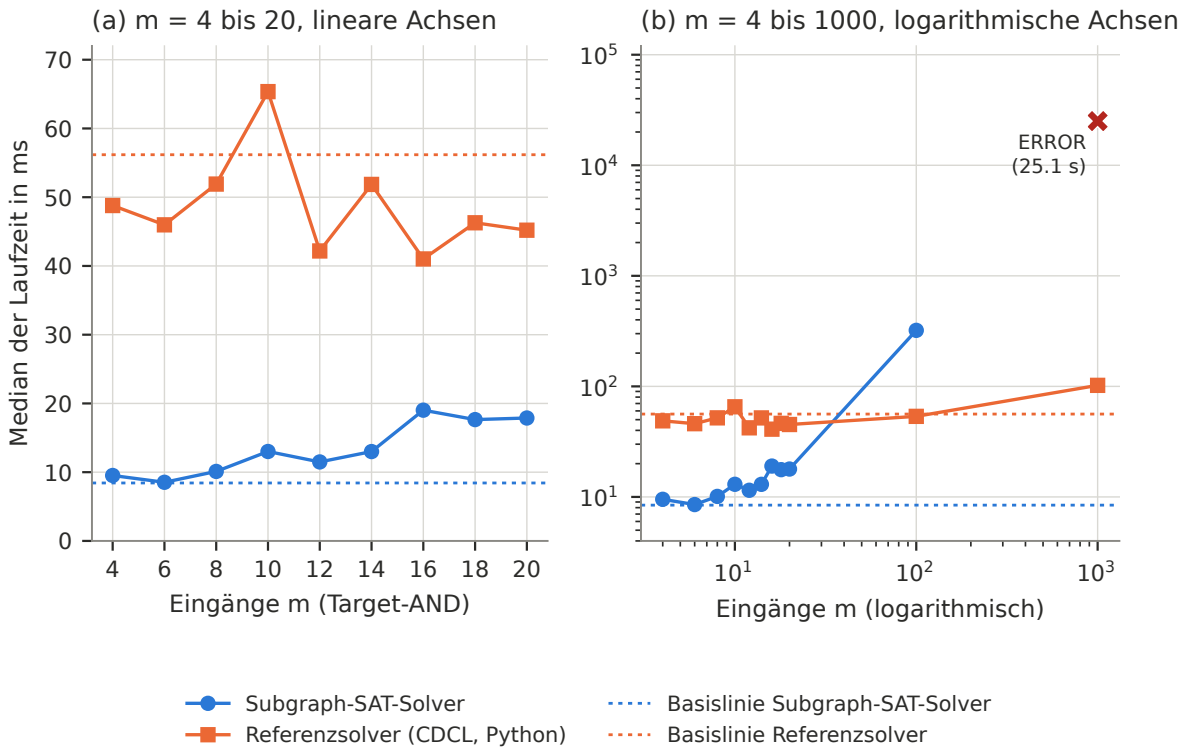


Abbildung 8: Median der Laufzeit der Target-AND-Circuits in Abhängigkeit von der Eingangszahl m, links für $m = 4$ bis 20 (lineare Achsen), rechts bis $m = 1000$ (logarithmische Achsen), für beide Solver.

bei $m = 6$ (8,53 ms), das Maximum bei $m = 16$ (19,02 ms). Eine lineare Regression über die neun Werte ergibt eine Steigung von 0,66 ms je zusätzlichem Eingang bei $R^2 = 0{,}836$; die Rangkorrelation nach Spearman beträgt $\rho = 0{,}883$ ($n = 9$). Der Anstieg ist klein gegen die Instanzgröße: Gates, Klauseln und Literale wachsen von $m = 4$ bis $m = 20$ um den Faktor 5,6 (7 → 39 Gates, 38 → 214 Literale), der Median aber nur um den Faktor 1,9. Abzüglich der Basislinie (8,43 ms) bleiben Nettozeiten zwischen 0,1 und 10,6 ms; für $m \leq 8$ liegen die Mediane höchstens 1,7 ms über der Basislinie und sind damit praktisch nicht von der Prozessstart-Grundlast zu unterscheiden. Der Median des Referenzsolvers dagegen zeigt keinen Anstieg: Er liegt zwischen 41,02 ms ($m = 16$) und 65,36 ms ($m = 10$), also um die Basislinie von 56,18 ms, und acht der neun Werte liegen darunter. Die Steigung beträgt $-0{,}41$ ms je Eingang bei $R^2 = 0{,}093$ und $\rho = -0{,}417$. Der Referenzsolver misst in diesem Bereich somit praktisch nur den Interpreterstart; die Rechenzeit für die Formel geht im Rauschen des Prozessstarts unter.

Lesart von Teilbild (b). Mit $m = 100$ (199 Gates, 498 Klauseln, 1 094 Literale) steigt der Median des Subgraph-SAT-Solvers auf 321,23 ms und liegt damit 6,0-mal über dem des Referenzsolvers (53,56 ms). Von $m = 20$ (17,88 ms, 214 Literale) auf $m = 100$ wächst die Laufzeit um den Faktor 18,0 bei einem Literal-Faktor von 5,1. Zieht man die Basislinie ab, ergibt sich ein Faktor 33,1 und, auf die Literalzahl bezogen, ein Exponent von etwa 2,1; das ist ein Wert in der Größenordnung des Exponenten, der in Abschnitt 6.4.5 aus den ISCAS-Messungen bestimmt wird, aber aus nur zwei Punkten gewonnen. Bei $m = 1000$ (1 999 Gates, 4 998 Klauseln, 10 994 Literale) scheitert der Subgraph-SAT-Solver mit `ERROR` nach 25,09 s; die Instanz ist aber nach Konstruktion erfüllbar (eindeutige Belegung), und der

Referenzsolver löst sie mit 102,17 ms korrekt als `SAT`. Das Programm markiert diesen Lauf deshalb als „unerwartetes Ergebnis“. Der Referenzsolver wächst von $m = 100$ auf $m = 1000$ nur von 53,56 auf 102,17 ms.

6.4.4 Experiment 2: Target-AND-Random, Variation der Tiefe

Bei festem $m = 10$ wächst der Circuit mit der Tiefe d von 27 Gates und 174 Literalen ($d = 1$) auf 54 Gates und 409 Literale ($d = 12$); Tabelle 15 und Abbildung 9 zeigen die Ergebnisse.

Instanz	Gates	Klauseln	Literale	Status	n_{SG}	$\tilde{t}_{\mathrm{SG}}$	n_{Ref}	$\tilde{t}_{\mathrm{Ref}}$
$d = 1$	27	76	174	SAT	30	14,59	30	45,36
$d = 2$	30	86	198	UNSAT	30	16,28	30	43,80
$d = 3$	33	97	227	UNSAT	30	20,35	30	44,82
$d = 4$	35	105	247	UNSAT	30	20,82	30	44,12
$d = 5$	38	114	268	UNSAT	30	25,75	30	49,10
$d = 6$	40	122	288	UNSAT	30	30,06	30	48,17
$d = 7$	42	129	307	SAT	30	32,15	30	42,28
$d = 8$	44	136	326	UNSAT	30	39,13	30	45,35
$d = 9$	47	147	357	UNSAT	30	37,32	30	49,13
$d = 10$	49	153	371	UNSAT	30	43,30	30	46,05
$d = 11$	52	162	392	SAT	30	47,68	30	53,58
$d = 12$	54	169	409	UNSAT	30	60,04	30	44,42

Tabelle 15: Target-AND-Random ($m = 10$): Struktur, Status und Median $\tilde{t}$ der Laufzeit in ms für beide Solver (Bezeichnungen wie in Tabelle 14).

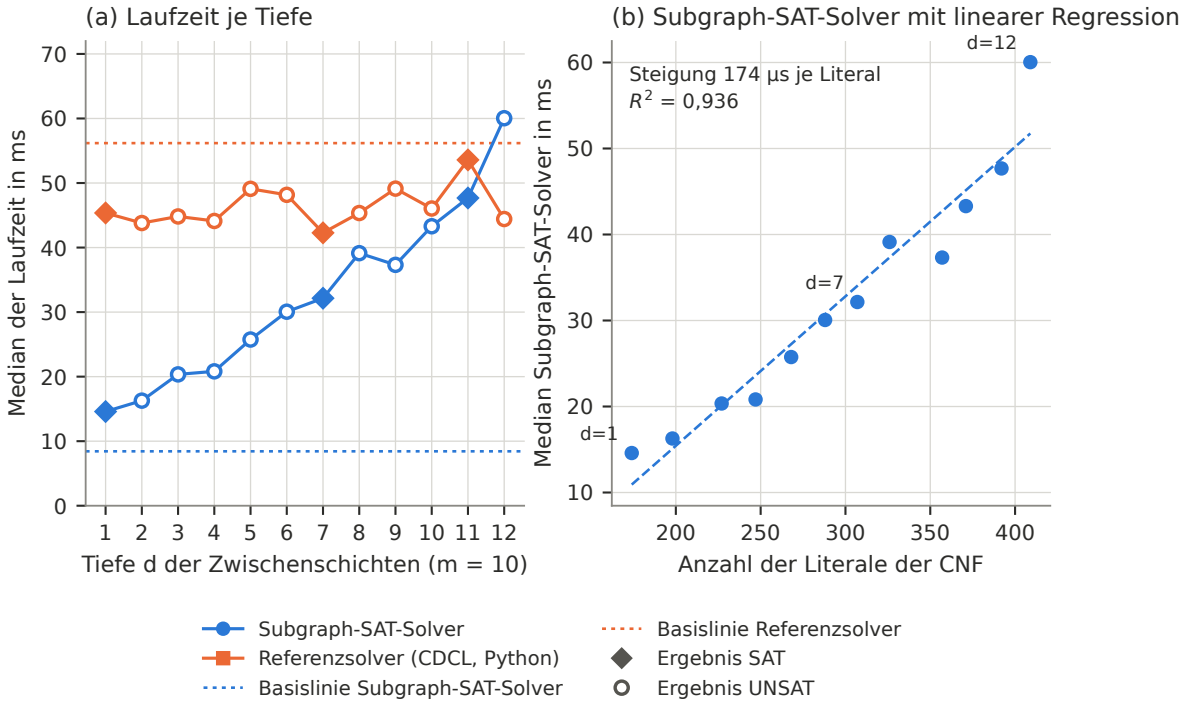


Abbildung 9: Median der Laufzeit der Target-AND-Random-Circuits ($m = 10$) über der Tiefe d für beide Solver mit Kennzeichnung von SAT und UNSAT (links) und über der Literalzahl mit linearer Regression für den Subgraph-SAT-Solver (rechts).

Aufbau der Abbildung. In Teilbild (a) von Abbildung 9 ist die Tiefe $d = 1, \ldots, 12$ auf der Abszisse und der Median in ms auf der Ordinate aufgetragen (Ordinate 0 bis 72 ms). Der blaue Linienzug gehört zum Subgraph-SAT-Solver, der orange zum Referenzsolver. Die Form der Marker kodiert das Ergebnis: Gefüllte Rauten bedeuten `SAT` ($d = 1, 7, 11$), weiße Kreise mit farbigem Rand bedeuten `UNSAT` (alle übrigen Tiefen). Beide Solver liefern bei jeder Tiefe dasselbe Ergebnis. Die gepunkteten Linien sind die Basislinien. Teilbild (b) trägt für den Subgraph-SAT-Solver den Median über der Literalzahl der CNF (174 bis 409) auf und zeigt die Regressionsgerade als gestrichelte Linie; im Bild sind die Steigung (174 μs je Literal) und das Bestimmtheitsmaß ($R^2 = 0{,}936$) angegeben. Die Punkte für $d = 1$, $d = 7$ und $d = 12$ sind beschriftet.

Lesart von Teilbild (a). Der Median des Subgraph-SAT-Solvers steigt mit der Tiefe nahezu monoton von 14,59 ms ($d = 1$) auf 60,04 ms ($d = 12$), also um den Faktor 4,1 bei einer Verdopplung der Gate-Zahl ($27 \to 54$) und einem Literal-Faktor von 2,35. Es gibt genau eine Abweichung von der Monotonie: Bei $d = 9$ (37,32 ms) liegt der Median unter dem von $d = 8$ (39,13 ms). Die Spearman-Korrelation zwischen Tiefe und Median beträgt $\rho = 0{,}993$ ($n = 12$), die lineare Regression ergibt 3,73 ms je Tiefenschicht bei $R^2 = 0{,}953$. Der Median des Referenzsolvers dagegen schwankt unsystematisch zwischen 42,28 ms ($d = 7$) und 53,58 ms ($d = 11$), die Steigung beträgt 0,33 ms je Schicht bei $R^2 = 0{,}145$ und $\rho = 0{,}343$. Er bleibt auf dem Niveau der Basislinie. Die Kurven kreuzen sich zwischen $d = 11$ (47,68 gegenüber 53,58 ms) und $d = 12$ (60,04 gegenüber 44,42 ms): Bei $d = 12$ ist der Subgraph-SAT-Solver erstmals langsamer als der Referenzsolver (1,35-faches).

Lesart von Teilbild (b). Die Punkte liegen eng an der Geraden mit der Steigung 0,174 ms je Literal. Die größten Abweichungen sind +8,29 ms bei $d = 12$, −5,39 ms bei $d = 9$, +3,67 ms bei $d = 1$ und −2,78 ms bei $d = 4$; alle übrigen Abweichungen betragen weniger als 2,0 ms. Das Verhältnis der Messwerte zur Geraden ist bei den `SAT`-Instanzen ($d = 1, 7, 11$: +3,67, −1,88, −1,11 ms) nicht anders als bei den `UNSAT`-Instanzen; in diesem Größenbereich ist also kein Unterschied zwischen erfüllbaren und unerfüllbaren Formeln erkennbar. Der Achsenabschnitt der Geraden (−19,3 ms) ist negativ; die Gerade beschreibt deshalb nur den Bereich der Messpunkte (174 bis 409 Literale) und nicht das Verhalten bei sehr kleinen Instanzen, wo die Basislinie dominiert. Bei den Selbst-Miter-Instanzen von Target-AND (`MiterSelf_TargetAND_m10`: 219 Literale, `MiterSelf_TargetAND_m20`: 439 Literale) liegen die Messwerte (19,97 und 53,58 ms) nahe an dem, was die Gerade für diese Literalzahlen ergibt (18,7 und 57,0 ms), obwohl es sich um andere, ebenfalls unerfüllbare Formeln handelt.

6.4.5 Experiment 3: ISCAS'85

Tabelle 16 und Abbildung 10 zeigen die zwölf ISCAS'85-Circuits mit 4 bis 3 512 Gates (26 bis 21 945 Literale).

Aufbau der Abbildung. Teilbild (a) von Abbildung 10 ist ein Balkendiagramm mit logarithmischer Ordinate (Median in ms, 3 bis 10^5). Die zwölf Circuits stehen in gleichen Abständen, aufsteigend nach Gate-Anzahl von c3 (4 Gates) bis c7552 (3 512 Gates); die Abszisse ist daher kategorial, und die Gate-Anzahl steht in Klammern hinter dem Namen. Je Circuit gibt es zwei Balken: blau für den Subgraph-SAT-Solver, orange für den

Instanz	Gates	Klauseln	Literale	Status	n_{SG}	$\tilde{t}_{\mathrm{SG}}$	n_{Ref}	$\tilde{t}_{\mathrm{Ref}}$
c3	4	12	26	SAT	30	6,98	30	45,19
c17	6	19	43	SAT	30	7,24	30	47,31
c432	160	515	1 259	SAT	30	515,75	30	71,23
c499	202	715	1 947	SAT	30	1 094,34	30	51,71
c880	383	1 113	2 571	SAT	30	2 167,29	30	47,61
c1355	546	1 611	3 739	SAT	13	3 945,39	30	48,84
c1908	880	2 379	5 375	SAT	6	9 931,22	30	52,76
c2670	1 193	3 270	7 422	SAT	3	20 427,66	30	70,09
c3540	1 669	4 609	10 487	ERROR / SAT	1	26 357,62	30	72,42
c5315	2 307	6 694	15 466	ERROR / SAT	1	29 415,04	30	67,58
c6288	2 416	7 217	16 817	ERROR / SAT	1	24 765,58	30	77,80
c7552	3 512	9 657	21 945	ERROR / SAT	1	40 737,40	30	89,88

Tabelle 16: ISCAS'85: Struktur, Status und Median $\tilde{t}$ der Laufzeit in ms für beide Solver, aufsteigend nach Gate-Anzahl (Bezeichnungen wie in Tabelle 14).

Referenzsolver. Rot schraffierte Balken mit weißer Füllung kennzeichnen ERROR-Läufe des Subgraph-SAT-Solvers; ihre Höhe ist die Zeit bis zum Fehler. Die gepunkteten Linien sind die Basislinien. Teilbild (b) trägt die Mediane der sechs gelösten Circuits c432, c499, c880, c1355, c1908 und c2670 doppelt-logarithmisch über der Literalzahl der CNF (1 259 bis 7 422) auf (blaue Punkte, beschriftet), dazu die angepasste Potenzfunktion als gestrichelte Gerade. Die Kreuze zeigen die vier fehlgeschlagenen Circuits (c3540 bis c7552) an ihrer Literalzahl; sie gehen nicht in den Fit ein. Ebenfalls nicht im Fit enthalten sind c3 und c17, deren Mediane (6,98 und 7,24 ms) unter der Basislinie liegen.

Lesart von Teilbild (a): Subgraph-SAT-Solver. Die blauen Balken wachsen von links nach rechts über drei Größenordnungen: c3 und c17 liegen bei 6,98 und 7,24 ms, c432 bei 515,75 ms, c499 bei 1 094,34 ms, c880 bei 2 167,29 ms, c1355 bei 3 945,39 ms, c1908 bei 9 931,22 ms und c2670 bei 20 427,66 ms. Dieses Wachstum ist durchgehend monoton. Die Zahl der Wiederholungen sinkt dabei mit der Laufzeit durch das Zeitbudget: $n = 30$ bis c880, $n = 13$ bei c1355, $n = 6$ bei c1908 und $n = 3$ bei c2670. Die Mediane der drei größten gelösten Circuits stützen sich also auf wenige Läufe. Die vier Circuits ab 1 669 Gates (c3540, c5315, c6288, c7552) enden mit ERROR nach 26,36, 29,42, 24,77 und 40,74 s (je ein Lauf, da bereits der Warm-up fehlschlug). Diese Zeiten sind keine Lösungszeiten.

Lesart von Teilbild (a): Referenzsolver. Die orangen Balken sind flach: von 45,19 ms (c3) bis 89,88 ms (c7552), der Median dieser zwölf Werte beträgt 60,17 ms, nahe der Basislinie. Der Referenzsolver findet für alle zwölf Circuits (mit Ausgang auf wahr) eine erfüllende Belegung. Eine leichte Zunahme mit der Größe ist erkennbar (die vier größten Circuits stehen mit 67,58 bis 89,88 ms am oberen Ende), sie beträgt aber nur wenige Dutzend Millisekunden.

Lesart von Teilbild (b). Die sechs Punkte liegen nahezu auf einer Geraden. Die Anpassung $t \propto L^b$ mit der Literalzahl L ergibt einen Exponenten $b = 2{,}08$ mit einem Standardfehler von 0,07 und $R^2 = 0{,}996$. Das 95 %-Konfidenzintervall der Steigung (t-Verteilung mit 4 Freiheitsgraden) beträgt rund [1,89; 2,27]. Der Exponent liegt damit in der Nähe von 2. Das ist mit der in Abschnitt 2.2 beschriebenen Struktur vereinbar, in der die Adjazenzmatrix des Clique-Graphen durch paarweise Vergleiche der Literalvorkommen aufgebaut wird und $O(L^2)$ Schritte benötigt; bewiesen ist dieser Zusammenhang durch die Messung aber nicht. Der Fit stützt sich auf nur sechs Punkte aus verschiedenen Cir-

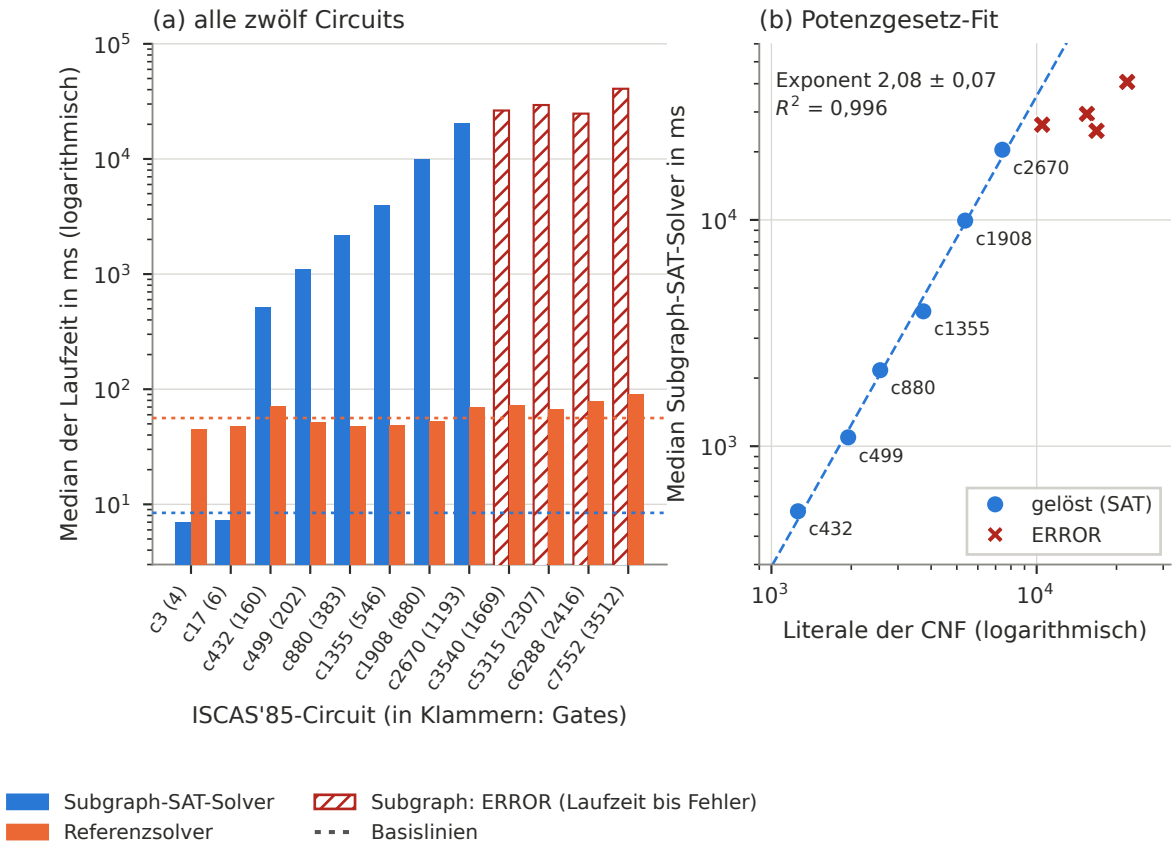


Abbildung 10: Median der Laufzeit der zwölf ISCAS'85-Circuits für beide Solver (links) und Potenzgesetz-Fit des Subgraph-SAT-Solvers über der Literalzahl der CNF für die sechs gelösten Circuits von c432 bis c2670 (rechts).

cuits (nicht auf eine Instanzfamilie mit einem Parameter) und auf Mediane, die bei c1355, c1908 und c2670 aus 13, 6 und 3 Läufen stammen. Er gilt außerdem nur für die gemessene Literalzahl der CNF und nicht für die tatsächliche Knotenzahl des Clique-Graphen (nach der Normalisierung auf höchstens drei Literale je Klausel sind Letztere etwas größer). Die Rechenzeit des externen Subgraph-Prozesses ist in diesem Wert nicht von der Reduktion getrennt (Abschnitt 2.2). Insbesondere liefert der Fit keine Bestätigung oder Widerlegung der theoretischen Schranke $\Theta(n^3)$ des Subgraph Algorithmus.

Der Ausfall ab 10 487 Literalen. Alle ISCAS-Circuits bis 7 422 Literale (c2670) werden gelöst; alle ab 10 487 Literalen (c3540) enden mit `ERROR`. Dieselbe Trennung zeigt der gesamte Datensatz: Alle sechs Instanzen mit `ERROR` haben mindestens 10 487 Literale (10 487, 10 994, 11 125, 15 466, 16 817 und 21 945), alle Instanzen mit höchstens 7 958 Literalen enden mit einem Ergebnis oder einem Timeout. Insbesondere fällt auch `TargetAND_m1000` aus, ein einfacher AND-Baum, den der Referenzsolver in 102 ms löst. Der Ausfall hängt also an der Instanzgröße und nicht an der logischen Schwierigkeit der Formel. Die Fehlerzeiten (24,77 bis 40,74 s) liegen deutlich unter dem Zeitlimit von 60 s; die Läufe endeten also nicht durch das Zeitlimit. Die Ursache wurde nicht untersucht. Die CSV-Datei des Zwischenstands enthält weder Rückgabecode noch Fehlermeldung (Abschnitt 6.3.4). Zu prüfen wären unter anderem der Speicherbedarf der Reduktion und des externen Subgraph-Prozesses auf der Maschine mit 3,9 GiB ohne Swap, die Größe

der JSON-Dateien mit den Kantenlisten sowie eine interne Fehlerbehandlung des Solvers (Abschnitt 6.2). Dass der Lauf des Full-Presets bei `TargetAND_m10000` (109 994 Literale) abbrach, passt zu dieser Größenschwelle, ist aber nicht belegt.

6.4.6 Miter-Instanzen: unerfüllbare Formeln

Die neun Miter-Instanzen sind unerfüllbar. Tabelle 17 und Abbildung 11 zeigen die Ergebnisse.

Instanz	Gates	Klauseln	Literale	Status	n_{SG}	$\tilde{t}_{\mathrm{SG}}$	n_{Ref}	$\tilde{t}_{\mathrm{Ref}}$
`MiterSelf_c17`	15	48	116	UNSAT	30	9,86	30	46,92
`MiterSelf_TargetAND_m10`	39	99	219	UNSAT	30	19,97	30	47,66
`MiterSelf_TargetAND_m20`	79	199	439	UNSAT	30	53,58	30	44,95
`MiterSelf_c432`	328	1 065	2 623	Timeout / UNSAT	1	60 041,35	30	279,73
`MiterSelf_c499`	437	1 590	4 374	Timeout / UNSAT	1	60 069,29	30	1 853,19
`Miter_c499_c1355`	781	2 486	6 166	Timeout / UNSAT	1	60 180,17	16	1 723,69
`MiterSelf_c880`	793	2 356	5 532	Timeout / UNSAT	1	60 114,38	19	3 293,14
`MiterSelf_c1355`	1 125	3 382	7 958	Timeout / UNSAT	1	60 273,65	7	3 139,56
`MiterSelf_c1908`	1 786	4 883	11 125	ERROR / UNSAT	1	25 559,73	15	3 374,42

Tabelle 17: Miter-Instanzen (alle unerfüllbar): Struktur, Status und Median $\tilde{t}$ der Laufzeit in ms für beide Solver, aufsteigend nach Gate-Anzahl (Bezeichnungen wie in Tabelle 14).

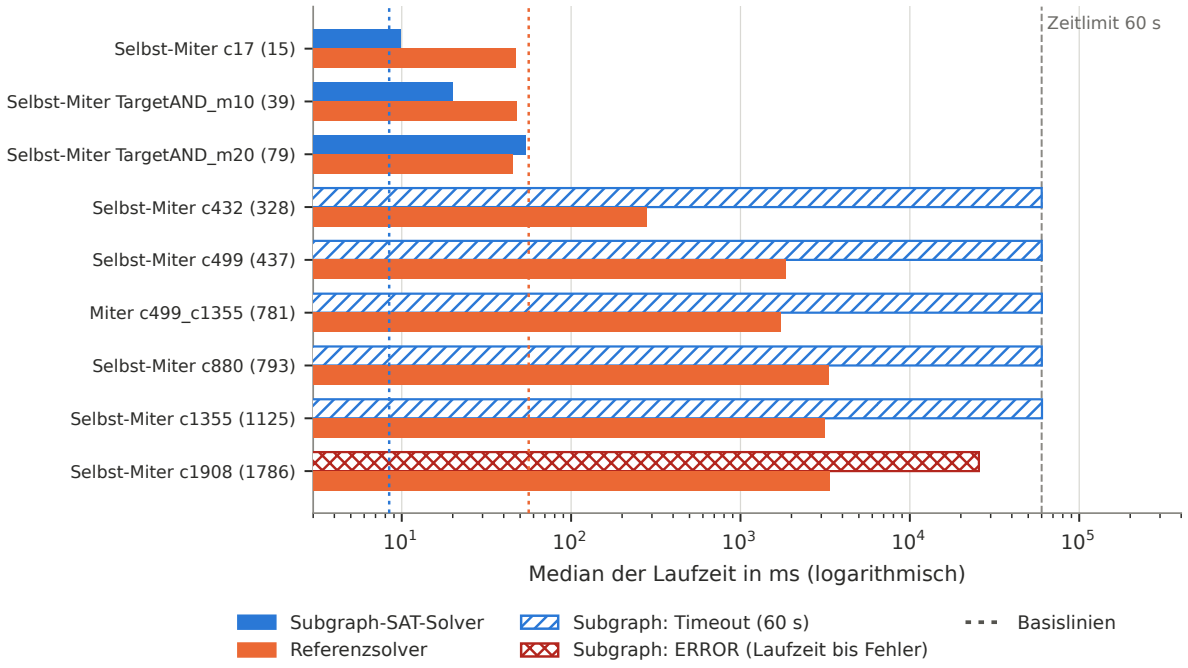


Abbildung 11: Median der Laufzeit der neun unerfüllbaren Miter-Instanzen für beide Solver mit Kennzeichnung der Timeouts und des ERROR-Laufs des Subgraph-SAT-Solvers.

Aufbau der Abbildung. Abbildung 11 ist ein horizontales, gruppiertes Balkendiagramm mit logarithmischer Abszisse (Median in ms, von 3 bis $4 \cdot 10^5$). Jede Instanz hat eine Zeile mit zwei Balken, oben blau für den Subgraph-SAT-Solver, unten orange für den Referenzsolver. Die Zeilen sind aufsteigend nach Gate-Anzahl (in Klammern) sortiert, die kleinste oben. Blau schraffierte Balken mit weißer Füllung kennzeichnen Timeouts des

Subgraph-SAT-Solvers; sie enden bei etwa 60 s, der senkrechten gestrichelten Linie. Der rot gekreuzte Balken der untersten Zeile ist der `ERROR`-Lauf bei `MiterSelf_c1908`. Die beiden gepunkteten senkrechten Linien sind die Basislinien.

Lesart. Der Subgraph-SAT-Solver löst nur die drei kleinsten Instanzen: `MiterSelf_c17` (15 Gates, 9,86 ms), `MiterSelf_TargetAND_m10` (39 Gates, 19,97 ms) und `MiterSelf_TargetAND_m20` (79 Gates, 53,58 ms), jeweils korrekt als `UNSAT`. Alle anderen sechs Instanzen scheitern: Fünf laufen in das Zeitlimit (`MiterSelf_c432`, `MiterSelf_c499`, `Miter_c499_c1355`, `MiterSelf_c880`, `MiterSelf_c1355`; Dauer des einen Laufs 60 041 bis 60 274 ms), und `MiterSelf_c1908` endet nach 25,56 s mit `ERROR` (nach Abschnitt 6.4.5 ist dies wegen der Größe von 11 125 Literalen zu erwarten). Der Referenzsolver löst dagegen *alle neun* Instanzen korrekt als `UNSAT`: 46,92, 47,66 und 44,95 ms für die drei kleinsten (auf Höhe der Basislinie), dann 279,73 ms (c432), 1 853,19 ms (c499), 1 723,69 ms (c499 gegen c1355), 3 293,14 ms (c880), 3 139,56 ms (c1355) und 3 374,42 ms (c1908). Die Mediane der größeren Miter-Instanzen sind nicht streng nach Gate-Anzahl geordnet: c880 mit 793 Gates (3,29 s) ist langsamer als c1355 mit 1 125 Gates (3,14 s). Wegen des Zeitbudgets stützen sich diese Mediane auf unterschiedlich viele Läufe ($n = 30$ bei c432 und c499, 19 bei c880, 16 bei c499 gegen c1355, 15 bei c1908 und 7 bei c1355).

Vergleich mit den erfüllbaren ISCAS-Instanzen. Beim Subgraph-SAT-Solver ist der Unterschied zwischen erfüllbaren und unerfüllbaren Instanzen bei vergleichbarer Literalzahl groß: `MiterSelf_c432` (2 623 Literale) erreicht das Zeitlimit von 60 s, während die erfüllbare Instanz c880 mit 2 571 Literalen nach 2,17 s gelöst wird, also mindestens 27,7-mal schneller. Ebenso liegen die übrigen vier Timeouts bei 4 374 bis 7 958 Literalen, wo c1355 (3 739 Literale), c1908 (5 375) und c2670 (7 422) noch in 3,9 bis 20,4 s gelöst werden. Aus diesen Daten allein lässt sich allerdings nicht entscheiden, ob dies an der Unerfüllbarkeit liegt oder an der Struktur der Miter-Formeln (die Miter-Formeln von c499, c880, c1355 und c1908 enthalten Klauseln mit 26 bis 33 Literalen, die ISCAS-Formeln höchstens 10; `MiterSelf_c432` hat wie c432 höchstens 10), denn beides ändert sich gleichzeitig. Bei den kleinen unerfüllbaren Instanzen (Target-AND-Random mit $d \geq 2$, Selbst-Miter von Target-AND) gibt es keine Auffälligkeit (Abschnitt 6.4.4).

6.4.7 Vergleich der beiden Solver

Abbildung 12 fasst das Verhältnis der Mediane beider Solver zusammen.

Aufbau der Abbildung. Abbildung 12 ist ein doppelt-logarithmisches Streudiagramm. Die Abszisse ist die Literalzahl der CNF (26 bis 21 945), die Ordinate der Quotient $\tilde{t}_{\mathrm{SG}}/\tilde{t}_{\mathrm{Ref}}$ der beiden Mediane. Die schwarze waagerechte Linie bei 1 markiert Gleichstand; Punkte darunter bedeuten, dass der Subgraph-SAT-Solver schneller war. Die Farbe und Form der Marker kodiert die Instanzfamilie: magentafarbene Kreise für Target-AND (dunkles Magenta für die Skalierungsinstanz $m = 100$), orange Rauten für Target-AND-Random, grüne Quadrate für ISCAS'85 und violette Dreiecke für Miter. Weiße Dreiecke mit Aufwärtspfeil sind Timeouts des Subgraph-SAT-Solvers; da der Lauf abgebrochen wurde, ist das gezeigte Verhältnis nur eine *untere* Schranke des wahren Verhältnisses. Die sechs `ERROR`-Instanzen sind nicht eingezeichnet, weil ihr Median keine Lösungszeit ist.

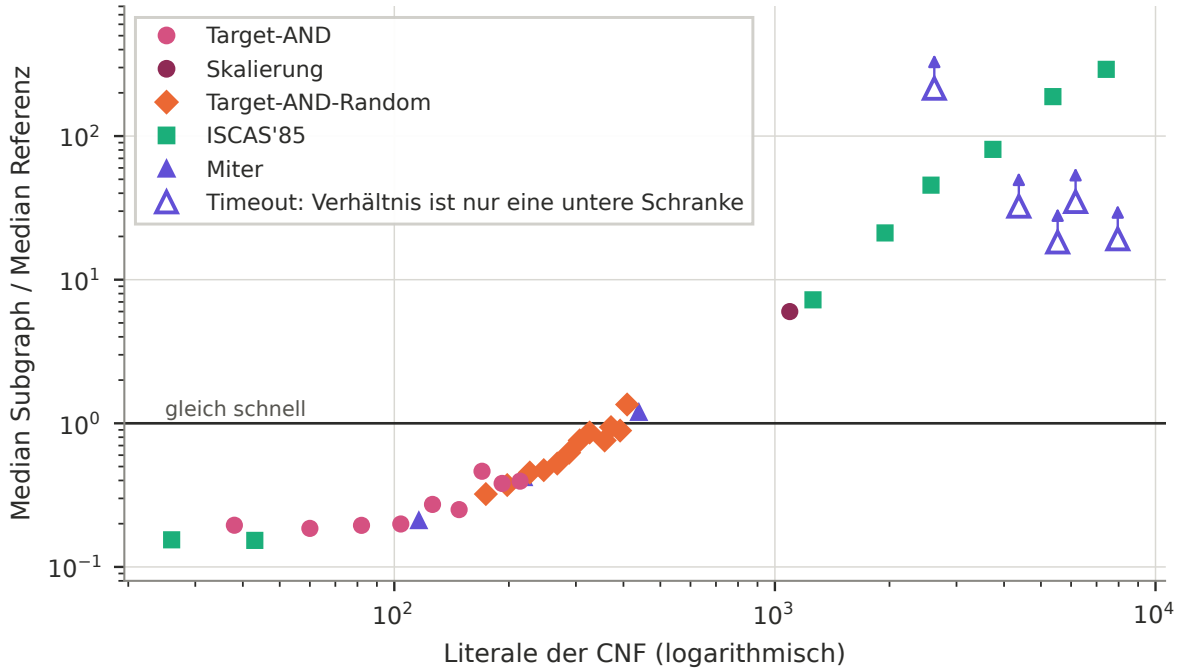


Abbildung 12: Verhältnis des Medians des Subgraph-SAT-Solvers zum Median des Referenzsolvers je Instanz über der Literalzahl der CNF (beide Achsen logarithmisch), farbig nach Instanzfamilie.

Lesart. Die Punkte steigen mit der Literalzahl von links unten nach rechts oben, weil der Referenzsolver kaum wächst (Abbildung 6), der Subgraph-SAT-Solver aber stark. Bei den 24 Instanzen mit höchstens 392 Literalen ist der Subgraph-SAT-Solver jeweils schneller (Quotient zwischen 0,15 bei c3 und c17 und 0,94 bei $d = 10$); ab 409 Literalen ($d = 12$: 1,35) ist er in allen Fällen langsamer: 1,19 bei `MiterSelf_TargetAND_m20`, 6,0 bei $m = 100$, 7,2 bei c432, 21,2 bei c499, 45,5 bei c880, 80,8 bei c1355, 188,2 bei c1908 und 291,5 bei c2670, dem größten gelösten Verhältnis. Die Timeouts liegen bei Quotienten von mindestens 18,3 bis mindestens 214,6. Die Kreuzung bei rund 400 Literalen ist vor allem ein Effekt der Startkosten (8,43 ms gegenüber 56,18 ms). Sie hängt daher an der Wahl eines Python-Programms als Referenz und besagt nicht, dass der Subgraph-SAT-Solver für kleine Formeln einen besseren Algorithmus hat.

6.4.8 Korrektheit und Übereinstimmung der Solver

Bei 33 Instanzen liefern beide Solver ein Ergebnis (`SAT` oder `UNSAT`); in allen 33 Fällen sind die Ergebnisse gleich. Dies sind alle 9 Target-AND-Circuits, alle 12 Target-AND-Random-Circuits, 8 ISCAS-Circuits (c3, c17, c432, c499, c880, c1355, c1908, c2670), die drei kleinen Miter-Instanzen und `TargetAND_m100`. Bei den 13 Instanzen dieser Menge, deren Ergebnis aus der Konstruktion folgt (Target-AND und `TargetAND_m100`: `SAT`; Miter: `UNSAT`), stimmen beide Solver mit der erwarteten Antwort überein. Der Referenzsolver trifft die erwartete Antwort bei allen 20 Instanzen mit bekanntem Ergebnis (Target-AND, Skalierung und Miter). Der Subgraph-SAT-Solver verfehlt sie bei zwei Instanzen, `MiterSelf_c1908` und `TargetAND_m1000`, jeweils mit `ERROR`; die Konsolenausgabe markiert beide als „unerwartetes Ergebnis“. Ein `ERROR` ist keine falsche Antwort, aber auch kein Ergebnis. Die Konsolenausgabe enthält keine Markierung „Modell ungültig“; für keine der ausgewer-

teten Instanzen wurde also gemeldet, dass ein ausgegebenes Modell die CNF oder den Circuit verletzt. Da die Modellprüfung nur für den ersten Lauf je Instanz und nur bei ausgegebenem Modell erfolgt und die Einzelergebnisse im Zwischenstand fehlen, kann dies nicht aus der CSV-Datei nachgeprüft werden.

6.5 Diskussion der neuen Version

6.5.1 Was die neuen Daten stützen

- **Die Messung enthält jetzt Skalierungsinformation.** Anders als in der ersten Messreihe, deren Werte für Instanzen von 4 bis 3 512 Gates gleich blieben, wächst der Median des Subgraph-SAT-Solvers deutlich mit der Instanzgröße: bei Target-AND-Random um den Faktor 4,1 zwischen 27 und 54 Gates, bei ISCAS'85 um mehr als drei Größenordnungen zwischen c17 und c2670. Die Basislinie von 8,43 ms ist so klein, dass Laufzeiten ab etwa 20 ms nicht mehr von ihr dominiert werden.

- **Näherungsweise quadratisches Wachstum im gemessenen Bereich.** Für die sechs gelösten ISCAS-Circuits von 1 259 bis 7 422 Literalen ergibt sich ein Exponent von $2{,}08 \pm 0{,}07$ (Standardfehler) über der Literalzahl.

- **Die Ergebnisse sind korrekt, soweit sie vorliegen.** In allen 33 Fällen, in denen beide Solver entscheiden, stimmen sie überein, und alle bekannten erwarteten Antworten werden getroffen.

- **Kein sichtbarer Unterschied zwischen SAT und UNSAT bei kleinen Instanzen.** Bei Target-AND-Random liegen `SAT`- und `UNSAT`-Instanzen auf derselben Regressionsgeraden.

6.5.2 Was die neuen Daten nicht stützen

- **Keine Aussage über die Schranke $\Theta(n^3)$.** Der Exponent 2,08 gilt für die Literalzahl der CNF und für die gesamte Pipeline (Reduktion, Datei-I/O, externer Prozess), nicht für den Subgraph Algorithmus allein (Abschnitt 2.2). Er stützt sich auf sechs Punkte und bei den größten drei auf 13, 6 und 3 Läufe.

- **Keine Aussage über etablierte SAT-Solver.** Der Referenzsolver ist ein Python-Programm ohne Anspruch auf Geschwindigkeit; sein Quotient zum Subgraph-SAT-Solver (Abbildung 12) ist kein Maß für den Vergleich mit MiniSat oder CaDiCaL. Ein Vergleich mit diesen Solvern fehlt weiterhin.

- **Keine Aussage über die Ursache der `ERROR`-Läufe und der Timeouts.** Rückgabecode, Fehlermeldung und Speichernutzung liegen im Zwischenstand nicht vor. Dass die Ausfälle an der Instanzgröße hängen (ab 10 487 Literalen), ist ein Befund der Daten; warum sie auftreten, ist offen. Dass unerfüllbare Instanzen für den Subgraph-SAT-Solver schwerer sind, lässt sich aus den Daten nicht von der Wirkung der langen Klauseln der Miter trennen.

- **Keine Aussagen über die Streuung.** Es liegen nur Mediane vor. Konfidenzintervalle, Quartile und die Einzelläufe sind nicht verfügbar. Unterschiede von wenigen Millisekunden zwischen zwei Instanzen, etwa die Nicht-Monotonie bei $d = 8$ und $d = 9$, lassen sich deshalb nicht von Messrauschen unterscheiden.

– **Keine Skalierung bis 10^5 Gates.** Die Skalierungsinstanzen $m = 10\,000$ und $m = 50\,000$ wurden nicht ausgewertet. Die größte gelöste Instanz hat 1 193 Gates.

6.5.3 Gültigkeitsbedrohungen

– **Interne Validität.** Ein Kern teilt sich zwischen Solver und Messprozess. Die Basislinie des Referenzsolvers hat eine Standardabweichung von 14,63 ms bei einem Median von 56,18 ms; die Mediane des Referenzsolvers unter etwa 70 ms sind deshalb nicht voneinander zu unterscheiden. Die Basislinie wurde nur einmal vor dem Preset gemessen, nicht laufend.

– **Konstruktvalidität.** Gemessen wird die Wallclock-Zeit eines Prozesses einschließlich Prozessstart. Die solverinterne Zeit wird im Zwischenstand nicht berichtet. Die Spalte `runtime_net_ms` (Laufzeit minus Basislinie je Lauf) liegt im Zwischenstand nicht vor; Nettozeiten in diesem Abschnitt sind Differenzen von Medianen.

– **Externe Validität.** Die Target-AND-Instanzen sind einfache AND-Bäume mit eindeutiger Belegung; die Target-AND-Random-Instanzen sind klein (27 bis 54 Gates). Industrielle Instanzen sind nur die ISCAS'85-Circuits mit bis zu 3 512 Gates; sie sind für moderne SAT-Solver klein. Die Ausgangsbedingung betrifft jeweils nur den ersten Ausgang.

– **Umgebung.** Eine virtuelle Maschine mit einer CPU, 3,9 GiB Speicher ohne Swap, Build ohne `-march=native` (Abschnitt 6.2). Die Ergebnisse gelten für diese Umgebung; schon der Speicher kann die Grenze von 10 487 Literalen bestimmen.

– **Unvollständiger Lauf.** Der Full-Preset wurde nicht abgeschlossen (Abschnitt 6.3.4). Die ausgewerteten 44 Instanzen sind vollständig, aber der Datensatz ist ein Zwischenstand.

6.5.4 Vergleich der Befunde mit der ersten Messreihe

Die Diskussion der ersten Messreihe hatte vermutet, dass deren konstante Laufzeiten ein Messartefakt aus Prozessstart und Zeitauflösung seien (Abschnitt 5). Die neue Version stützt diese Vermutung in drei Punkten: (i) Mit nanosekundengenauer Zeitmessung liegen die Mediane nicht mehr auf den Zeittakten bei 15,625 und 31,25 ms (die Einzelläufe sind im Zwischenstand nicht verfügbar). (ii) Die Basislinie der trivialen Instanz liegt mit 8,43 ms in der Größenordnung der kleinsten Messwerte kleiner Instanzen, und erst oberhalb davon steigen die Mediane mit der Instanzgröße. (iii) Bei größeren Instanzen wächst die Laufzeit des Subgraph-SAT-Solvers deutlich. Die in der ersten Messreihe beobachtete Unabhängigkeit der Laufzeit von der Instanzgröße war demnach keine Eigenschaft des Solvers. Der absolute Vergleich ist wegen der unterschiedlichen Instanzen (korrigierte Kodierung, Ausgangsbedingung) und der nicht dokumentierten Umgebung der ersten Messreihe nicht möglich (Abschnitt 6.1).

Die erste Messreihe fand außerdem keine Solver-Ausgabe. Die neue Version zeigt, dass der Subgraph-SAT-Solver bei den kleinen und mittleren Instanzen ein Ergebnis liefert, das mit dem Referenzsolver übereinstimmt.

6.5.5 Offene Punkte

1. Den Full-Preset vollständig wiederholen und dabei `experiment_results.csv`, `summary.csv`, `environment.json`, `baseline.json` und Rückgabecodes sichern; insbesondere klären, warum der Lauf bei `TargetAND_m10000` abbrach.

2. Die Ursache der `ERROR`-Läufe ab 10 487 Literalen untersuchen (Rückgabecode, Fehlermeldung, Speichernutzung `peak_memory_mb`) und die Skalierungsinstanzen $m = 10\,000$ und $m = 50\,000$ auf einer Maschine mit mehr Speicher messen.

3. Den Solver mit CMake (`-O3 -march=native`) bauen und messen; die Version des Solvers festhalten.

4. Den Referenzsolver durch MiniSat oder CaDiCaL ergänzen, damit der Vergleich nicht mehr durch die Startzeit des Python-Interpreters bestimmt wird.

5. Mehr Messpunkte zwischen 1 000 und 10 000 Literalen (z. B. Target-AND mit $m = 200$ bis 800), um den Exponenten an einer Instanzfamilie mit einem Parameter zu prüfen, und Instanzen mit gleicher Literalzahl, aber unterschiedlicher Klauselänge, um den Einfluss der Klauseln der Miter von dem der Unerfüllbarkeit zu trennen.

6.6 Reproduktion der neuen Auswertung

Die Rohdaten des Zwischenstands liegen unter `src/results/`:

Listing 2: Dateien der neuen Version 2.0.0 (Zwischenstand)

```
src/results/
  full_zwischenstand.csv       Median je Solver und Instanz (88
     Zeilen)
  full_log_zwischenstand.log   Konsolenausgabe des Full-Laufs (
     mit Basislinien)
  hardware.md                  Hardware und Software der
     Messungen
  paper/                       fig_v2_*.pdf, tab_v2_*.tex,
     summary_v2.json
```

Die Abbildungen und Tabellen dieses Abschnitts erzeugt das Skript `src/paper_analysis_v2.py`:

Listing 3: Erzeugung der Auswertung der neuen Version

```
pip install -e .
python -m src.paper_analysis_v2
cd science && pdflatex subgraph-sat-solver-experiments.tex
```

Das Skript berechnet die Struktur der Instanzen (Variablen, Klauseln, Literale) mit den Generatoren des Frameworks neu, prüft sie gegen die gemessenen Gate-Zahlen und schreibt alle im Text genannten Kennzahlen (Regressionen, Korrelationen, Exponent, Statuszähler) nach `src/results/paper/summary_v2.json`. Ein vollständiger neuer Lauf wird mit `python-msrc.run_experiments--presetfull--solver-path<Pfad> --with-reference` gestartet.

7 Zusammenfassung

Wir haben alle 288 Läufe der vier Experiment-Suites des Subgraph-SAT-Solver-Frameworks ausgewertet. Die Laufzeiten liegen zwischen 6,02 und 40,27 ms und sind über Instanzgrößen von 4 bis 3 512 Gates, über alle drei Circuit-Typen und über alle vier Suites statistisch nicht unterscheidbar. Ein Zusammenhang mit Gate-Anzahl ($\rho = 0{,}024$), Tiefe ($\rho = -0{,}086$) oder Eingangszahl liegt nicht vor. Die Messwerte häufen sich bei 15,625 und 31,25 ms, und in keinem Lauf wurde ein Solver-Ergebnis erfasst. Zudem kodiert der CNF-Encoder die Gate-Typen NAND, NOR und XOR fehlerhaft, was die ISCAS'85-Formeln betrifft. Die Messungen sind daher als Obergrenze für die Prozesslaufzeit zu lesen (der Solver-Prozess endet für alle Instanzen ohne Timeout in unter 41 ms), nicht als Skalierungsstudie und nicht als Nachweis korrekter Lösungen. Die in Abschnitt 5 beschriebenen Änderungen am Messaufbau sind Voraussetzung dafür, das Skalierungsverhalten des Solvers und die theoretische $\Theta(n^3)$-Schranke empirisch zu prüfen.

7.1 Ausblick

- Wiederholung aller Experimente mit dem verbesserten Messaufbau.
- Analyse von Strukturmaßen der ISCAS-Instanzen (Tiefe, Fan-out, Clustering) und ihres Einflusses auf die Solver-Zeit.
- Untersuchung unerfüllbarer Instanzen sowie Worst-Case-Konstruktionen für den Subgraph Algorithmus.
- Vergleich mit klassischen SAT-Solvern auf identischen Eingaben.

7.2 Implementierungshinweis

Der Code und die Rohdaten sind verfügbar unter: https://github.com/naphez86/subgraph-sat-solver-experiments.

A Reproduzierbarkeit und Datenquellen

Alle Rohdaten liegen unter `src/results/<suite>/`:

Listing 4: Ergebnisstruktur

```
src/results/
  quick/        experiment_results.{json,csv}   21 Laeufe
  standard/     experiment_results.{json,csv}   33 Laeufe
  full/         experiment_results.{json,csv}  117 Laeufe
  performance/  experiment_results.{json,csv}  117 Laeufe
  paper/        Tabellen (.tex), Abbildungen (.pdf), summary.json
```

Tabellen und Abbildungen dieser Arbeit werden aus den CSV-Dateien erzeugt:

Listing 5: Erzeugung der Auswertung

```
pip install -e ".[paper]"
python -m src.paper_analysis
cd science && pdflatex subgraph-sat-solver-experiments.tex
```

Das Skript schreibt nach `src/results/paper/`; die Kennzahlen im Text (Korrelationen, Testwerte, Anteile) sind zusätzlich in `summary.json` abgelegt.

B Beispiel-Datensätze

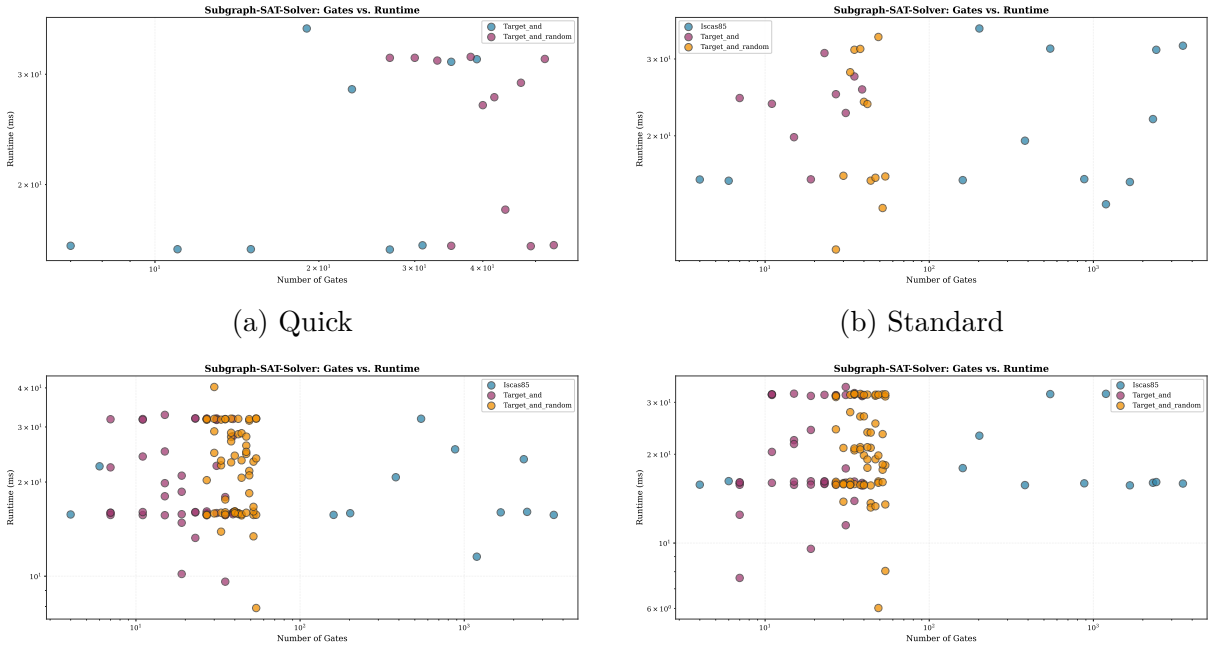
Die folgenden Zeilen stammen unverändert aus `src/results/full/experiment_results.csv` (Spalten gekürzt):

Listing 6: Ausgewaehlte Datensaetze der Full-Suite

```
circuit_type,circuit_name,num_inputs,num_gates,num_clauses,satisfiable,runtime_ms,timestamp
target_and,TargetAND_m4,4,7,20,False,22.2797,2026-07-14 09:40:24
target_and_random,TargetAND_m10_d1,10,27,91,False,15.6600,2026-07-14 09:40:24
iscas85,c7552,207,3512,10690,False,15.7025,2026-07-14 09:40:27
```

C Diagramme der einzelnen Suites

Das Framework erzeugt für jede Suite sechs Diagramme (Laufzeit gegen Größe, Verteilung, Gates gegen Laufzeit, Klausel-Histogramm, Leistungsvergleich, Skalierungsanalyse) als SVG und PDF unter `src/results/<suite>/plots/`. Abbildung 13 zeigt exemplarisch das Streudiagramm „Gates gegen Laufzeit“ aller vier Suites; die Abbildungen der anderen Diagrammtypen und die zugehörigen Textberichte (`report.txt`) liegen im selben Ordner.



(a) Quick (b) Standard

(c) Full (d) Performance

Abbildung 13: Gates gegen Laufzeit je Suite (vom Framework erzeugte Diagramme). Beide Achsen sind logarithmisch; Achsenbereiche und Farbzuordnung werden je Teilbild automatisch gewählt (in (a) blau: Target_and, magenta: Target_and_random; in (b)–(d) blau: Iscas85, magenta: Target_and, orange: Target_and_random).

Gemeinsame Merkmale der vier Teilbilder. Jedes Teilbild trägt den Titel „Subgraph-SAT-Solver: Gates vs. Runtime“ und zeigt die Anzahl der Gates (Abszisse) gegen die Laufzeit in ms (Ordinate). Anders als in Abbildung 4 sind *beide* Achsen logarithmisch; die Ordinate ist mit Beschriftungen wie 2×10^1 oder 3×10^1 versehen, sodass Abstände auf ihr nicht linear zu lesen sind (eine Verdopplung von 15,625 auf 31,25 ms entspricht überall dem gleichen Abstand). Jeder Lauf ist ein großer, leicht transparenter Punkt ohne horizontale Verschiebung; Läufe mit identischer Gate-Anzahl und nahezu gleicher Laufzeit verdecken sich daher gegenseitig. Die Legende verwendet die Rohbezeichnungen des Frameworks. Zu beachten ist, dass die Farben je Teilbild vergeben werden und deshalb nicht überall demselben Typ entsprechen (siehe Bildunterschrift). Die Achsenbereiche sind ebenfalls je Teilbild automatisch gewählt, die vier Teilbilder sind also nicht unmittelbar maßstabsgleich.

(a) Quick. Das Teilbild enthält 21 Punkte: 9 Target-AND- (blau) und 12 Target-AND-Random-Läufe (magenta) zwischen 7 und 54 Gates. Es gibt keinen ISCAS-Lauf, daher endet die Abszisse bei etwa 54 Gates. Die Laufzeiten reichen von 15,75 bis 35,50 ms. Die Punkte ordnen sich in zwei Reihen an: acht Punkte bei etwa 15,8 bis 16 ms am unteren Rand, sieben bei etwa 31,4 bis 32 ms im oberen Bereich; zusammen sind das 15 von 21 Läufen (71,4 %) innerhalb von ±1 ms um die beiden Zeittakte. Von den verbleibenden sechs Punkten liegen fünf dazwischen (18,24 bis 29,08 ms); der sechste ist der Höchstwert 35,50 ms bei 19 Gates. Weil jede Instanz nur einmal läuft, gibt es keine senkrechten Stapel. Ein Anstieg zu größeren Gate-Zahlen ist nicht zu erkennen; der höchste Punkt gehört zu einem der kleineren Circuits.

(b) Standard. Mit 33 Läufen kommen erstmals die 12 ISCAS-Circuits hinzu (blau), wodurch die Abszisse bis 3 512 Gates reicht. Die Laufzeiten liegen zwischen 10,97 (27 Gates, Target-AND-Random) und 35,24 ms (c499 mit 202 Gates). Die ISCAS-Punkte verteilen sich über den gesamten Wertebereich ohne Anstieg zu den großen Circuits: Der Median der ISCAS-Läufe beträgt 17,70 ms, der kleinste Wert 13,93 ms; am unteren Band stehen sowohl sehr kleine (c3, c17) als auch große Circuits. 16 von 33 Läufen (48,5 %) liegen innerhalb von ±1 ms um die Zeittakte, der geringste Anteil aller vier Suites.

(c) Full. Dieses Teilbild enthält 117 Läufe, davon 12 ISCAS-Läufe. Hier wird jeder synthetische Circuit fünfmal ausgeführt, die fünf Läufe haben dieselbe Gate-Anzahl und bilden senkrechte Stapel. Besonders dicht ist die Säule der Target-AND-Random-Circuits (orange) zwischen 27 und 54 Gates, in der sich zahlreiche Punkte überdecken; sichtbar sind deshalb mehr die beiden Bänder bei 16 und 32 ms als die Einzelpunkte. Am oberen Rand steht bei 30 Gates der Einzelwert 40,27 ms, am unteren Rand bei 54 Gates der Einzelwert 7,91 ms; beide stammen von Target-AND-Random-Circuits ($d = 2$ bzw. $d = 12$). Mindestens die Hälfte der ISCAS-Läufe liegt bei höchstens 16,03 ms (Median), also am unteren Band. 69 von 117 Läufen (59,0 %) liegen nahe den Zeittakten.

(d) Performance. Aufbau und Besetzung entsprechen (c). Die Ordinate beginnt tiefer (bei etwa 6 ms), weil hier der kleinste Wert der gesamten Studie liegt: 6,02 ms bei 49 Gates (Target-AND-Random, $d = 10$). Daneben fallen weitere Werte unter 10 ms auf (7,62 ms bei 7 Gates, 8,04 ms bei 54 Gates). Nach oben endet die Punktwolke bei 33,83 ms; es gibt also keinen Ausreißer wie in (c). Der Median der ISCAS-Läufe liegt bei 16,06 ms; mindestens

die Hälfte der blauen Punkte steht damit am unteren Band. 73 von 117 Läufen (62,4 %) liegen nahe den Zeittakten, der höchste Anteil der vier Suites.

Vergleich der Teilbilder. In allen vier Teilbildern zeigt sich dasselbe Muster aus zwei horizontalen Bändern bei etwa 15,6 bis 16 ms und 31 bis 32 ms, das unabhängig von Gate-Anzahl und Circuit-Typ auftritt. Mit wachsender Zahl der Läufe von (a) nach (c) und (d) wird dieses Muster deutlicher, weil mehr Punkte auf den Bändern liegen. Der Anteil der Läufe nahe den Zeittakten (71,4, 48,5, 59,0 und 62,4 %) schwankt zwischen den Suites, folgt aber keiner Ordnung. Dieselbe Zeitauflösung ist demnach in allen Suites wirksam.

Literatur

[1] S. Epp. *Der Subgraph Algorithmus*. Wissenschaftliche Arbeit, 2026. https://github.com/naphez86/subgraph.

[2] S. Epp. *Learning M-DNF in Boolean Circuits*. Abschlussarbeit (Masterarbeit) im Masterstudiengang Informatik, Universität Paderborn, 2013.

[3] F. Brglez, H. Fujiwara. A neutral netlist of 10 combinational benchmark circuits and a target translator in Fortran. *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 1985.

[4] G. S. Tseitin. On the complexity of derivation in propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic, Part II*, 1968.

[5] S. A. Cook. The complexity of theorem-proving procedures. *Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC)*, 1971.

[6] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

[7] L. A. Levin. Universal search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.

[8] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[9] C. H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.

[10] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[11] M. Sipser. *Introduction to the Theory of Computation*. 3rd edition, Cengage Learning, 2012.

[12] R. E. Ladner. On the structure of polynomial time reducibility. *Journal of the ACM*, 22(1):155–171, 1975.

[13] L. Berman and J. Hartmanis. On isomorphisms and density of NP and other complete sets. *SIAM Journal on Computing*, 6(2):305–322, 1977.

[14] S. A. Cook and R. A. Reckhow. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44(1):36–50, 1979.

[15] M. Davis and H. Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, 1960.

[16] M. Davis, G. Logemann, and D. Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962.

[17] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.

[18] T. J. Schaefer. The complexity of satisfiability problems. In *Proceedings of the 10th Annual ACM Symposium on Theory of Computing (STOC)*, pages 216–226, 1978.

[19] B. Monien and E. Speckenmeyer. Solving satisfiability in less than 2^n steps. *Discrete Applied Mathematics*, 10(3):287–295, 1985.

[20] choning1999 U. Schöning. A probabilistic algorithm for k-SAT and constraint satisfaction problems. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 410–414, 1999.

[21] R. Impagliazzo and R. Paturi. On the complexity of k-SAT. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.

[22] R. Paturi, P. Pudlák, M. E. Saks, and F. Zane. An improved exponential-time algorithm for k-SAT. *Journal of the ACM*, 52(3):337–364, 2005.

[23] J. P. Marques-Silva and K. A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[24] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference (DAC)*, pages 530–535, 2001.

[25] N. Eén and N. Sörensson. An extensible SAT-solver. In E. Giunchiglia and A. Tacchella, editors, *Theory and Applications of Satisfiability Testing (SAT 2003)*, Lecture Notes in Computer Science, volume 2919, pages 502–518. Springer, 2004.

[26] D. A. Plaisted and S. Greenbaum. A structure-preserving clause form translation. *Journal of Symbolic Computation*, 2(3):293–304, 1986.

[27] A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors. *Handbook of Satisfiability*. Frontiers in Artificial Intelligence and Applications, volume 185. IOS Press, 2009.